Competitive Programming Notebook

August 10, 2026

Contents

1 Miscellaneous

1.1 Template

```cpp
#include <bits/stdc++.h>

using namespace std;

//#define int long long
#define ll long long
#define ld long double
#define fi first
#define se second
#define pb push_back
#define eb emplace_back
#define all(x) x.begin(), x.end()
#define rall(x) x.rbegin(), x.rend()
#define ones(n) __builtin_popcountll(n)
#define msb(n) (63 - __builtin_clzll(n))
#define lsb(n) __builtin_ctzll(n)

const int N = 2e5 + 5, M = 1e3 + 5, LOG = 31;
const int inf = 0x3f3f3f3f;
const ll llinf = 0x3f3f3f3f3f3f3f3f;
const int MOD = 1e9 + 7;
const double eps = 1e-9, PI = acos(-1);

void testCase() {
}


void preCompute() {
}

int32_t main() {
#ifdef C_Lion
    freopen("input.txt", "r", stdin);
    freopen("output.txt", "w", stdout);
    freopen("errors.txt", "w", stderr);
#else
    //    freopen("input.in", "r", stdin);
    //    freopen("output.out", "w", stdout);
#endif

    ios_base::sync_with_stdio(false), cin.tie(nullptr), cout.tie(
         nullptr);
    cout << fixed << setprecision(static_cast<int32_t>(-log10(eps)));
    preCompute();

    int tc = 1;
    cin >> tc;
    for (int TC = 1; TC <= tc; TC++) {
        // cout << "Case #" << TC << ": ";
        testCase();
    }
}

/*



*/
```

1.2 fastIO

```cpp
const int MOD = 1e9 + 7;
const int BUF_SZ = 1 << 15;

inline namespace Input {
    char buf[BUF_SZ];
    int pos;
    int len;
    char next_char() {
        if (pos == len) {
            pos = 0;
            len = (int)fread(buf, 1, BUF_SZ, stdin);
            if (!len) { return EOF; }
        }
        return buf[pos++];
    }

    int read_int() {
        int x;
        char ch;
        int sgn = 1;
        while (!isdigit(ch = next_char())) {
            if (ch == '-') { sgn *= -1; }
        }
        x = ch - '0';
        while (isdigit(ch = next_char())) { x = x * 10 + (ch - '0'); }
        return x * sgn;
    }
}  // namespace Input
inline namespace Output {
    char buf[BUF_SZ];
    int pos;

    void flush_out() {
        fwrite(buf, 1, pos, stdout);
        pos = 0;
    }

    void write_char(char c) {
        if (pos == BUF_SZ) { flush_out(); }
        buf[pos++] = c;
    }

    void write_int(int x) {
        static char num_buf[100];
        if (x < 0) {
            write_char('-');
            x *= -1;
        }
        int len = 0;
        for (; x >= 10; x /= 10) { num_buf[len++] = (char)('0' + (x %
             10)); }
        write_char((char)('0' + x));
        while (len) { write_char(num_buf[--len]); }
        write_char('\n');
    }

    // auto-flush output when program exits
    void init_output() { assert(atexit(flush_out) == 0); }
}  // namespace Output
```

1.3 grid

```cpp
int dx[] = {0, 0, -1, 1, -1, -1, 1, 1};
int dy[] = {-1, 1, 0, 0, -1, 1, -1, 1};
char di[] = {'L', 'R', 'U', 'D'};
```

1.4 random

```cpp
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
template <typename T>
T random(T l, T r) {
    return uniform_int_distribution(l, r)(rng);
}

vector<int> random(int n) {
    vector<int> v(n);
    // shuffle 1
    shuffle(v.begin(), v.end(), rng);
    // shuffle 2
    for (int i = 1; i < n; i++)
        swap(v[i], v[uniform_int_distribution<int>(0, i)(rng)]);
    return v;
}
```

1.5 coordinateCompression

```cpp
// Coordinate Compression - O(N log N)
template<typename T>
struct Compress {
    vector<T> v;

    Compress(vector<T>& a) : v(a) {
        sort(all(v));
        v.erase(unique(all(v)), v.end());
        for (auto &x : a) {
            x = lower_bound(all(v), x) - v.begin();
        }
    }

    T operator[](int i) const {
        return v[i];
    }
};
```

1.6 grayCode

```cpp
// Gray Code Generator - O(2^N)
// g(n) = n ^ (n >> 1)
vector<int> grayCode(int n) {
    vector<int> res(1 << n);
```

```
    for (int i = 0; i < (1 << n); i++) {
        res[i] = i ^ (i >> 1);
    }
    return res;
}
```

1.7 customHashHashTable

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

// Custom Hash to prevent anti-hash tests in unordered_map / gp_hash_table
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM = chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};

// Usage: gp_hash_table<ll, int, custom_hash> table;
//        unordered_map<ll, int, custom_hash> safe_map;
```

1.8 pragmas

```
// GCC Optimizations
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

1.9 int128

```
// __int128 Stream Operators Helper
typedef __int128_t int128;
typedef __uint128_t uint128;

ostream &operator<<(ostream &os, int128 n) {
    if (n == 0) return os << "0";
    if (n < 0) { os << "-"; n = -n; }
    string s;
    while (n > 0) { s += (char)('0' + (n % 10)); n /= 10; }
    reverse(all(s));
    return os << s;
}

istream &operator>>(istream &is, int128 &n) {
    string s; is >> s;
    n = 0; int i = 0, sign = 1;
    if (!s.empty() && s[0] == '-') { sign = -1; i = 1; }
    for (; i < (int)s.size(); i++) n = n * 10 + (s[i] - '0');
    n *= sign;
    return is;
}
```

2 Data Structures (DS)

2.1 Segment Tree

2.1.1 SegmentTree

```
struct SEG {
    ll sum = 0;
    SEG() {
    }
    SEG(ll x){
        sum = x;
    }
};
struct segTree {
    vector<SEG> seg;
    int sz = 1,n;
    segTree(int nn){
        n = nn;
        while (sz < nn)
            sz *= 2;
        seg.assign(2 * sz , SEG());
    }

    SEG merge(SEG seg1, SEG seg2) {
        SEG ret;
        ret.sum = seg1.sum + seg2.sum;
        return ret;
    }

    void build(vector<int> &v,int x, int lx, int rx) {
        if (lx == rx) {
            seg[x] = SEG(v[lx]);
            return;
        }
        int mid = (lx + rx) / 2;
        int lf = 2 * x + 1, rt = 2 * x + 2;

        build(v,lf, lx, mid);
        build(v,rt, mid + 1, rx);
        seg[x] = merge(seg[lf], seg[rt]);
    }
    void build(vector<int> &v){
        build(v,0, 0, n-1);
    }

    void update(int i, ll val, int x, int lx, int rx) {
        if (lx == rx) {
            seg[x] = SEG(val);
            return;
        }
        int mid = (lx + rx) / 2,lf = 2*x+1,rt= 2*x+2;
        if(i <= mid)
            update(i, val, lf, lx, mid);
        else
            update(i, val, rt, mid + 1, rx);
        seg[x] = merge(seg[lf] , seg[rt]);
    }

    void update(int i, ll val) {
        update(i, val, 0, 0, n-1);
    }

    SEG query(int l, int r, int x, int lx, int rx) {
        if (l <= lx && r >= rx)
            return seg[x];
        if (l > rx || r < lx)
            return SEG();

        int mid = (lx + rx) / 2,lf = 2 * x + 1,rt = 2*x+2;

        return merge(query(l, r, lf, lx, mid) , query(l, r, rt, mid + 1, rx));
    }

    SEG query(int l, int r) {
        return query(l, r, 0, 0, n-1);
    }
};
```

2.1.2 LazySegmentTree

```
struct SEG {
    ll sum = 0, lz = 0;

    SEG() {
    }

    SEG(ll x) {
        sum = x;
    }
};

struct segTree {
    vector<SEG> seg;
    int sz = 1, n;
    ll NO_OP = 0;

    segTree(int nn) {
        n = nn;
        while (sz < nn)
            sz *= 2;
        seg.assign(2 * sz, SEG());
    }

    SEG merge(SEG seg1, SEG seg2) {
        SEG ret;
        ret.sum = seg1.sum + seg2.sum;
        return ret;
    }

    void build(vector<int> &v, int x, int lx, int rx) {
        if (lx == rx) {
            seg[x] = SEG(v[lx]);
            return;
        }
        int mid = (lx + rx) / 2;
        int lf = 2 * x + 1, rt = 2 * x + 2;

        build(v, lf, lx, mid);
        build(v, rt, mid + 1, rx);
        seg[x] = merge(seg[lf], seg[rt]);
    }

    void build(vector<int> &v) {
        build(v, 0, 0, n - 1);
    }

    void push(int x, int lx, int rx) {
        if (seg[x].lz != NO_OP) {
            seg[x].sum += seg[x].lz * (rx - lx + 1);
            int lf = 2 * x + 1, rt = 2 * x + 2;
            if (lx != rx) {
                seg[lf].lz += seg[x].lz;
                seg[rt].lz += seg[x].lz;
            }
            seg[x].lz = NO_OP;
        }
    }

    void update(int l, int r, ll val, int x, int lx, int rx) {
        push(x, lx, rx);
        if (l > rx || r < lx)
            return;
        if (l <= lx && r >= rx) {
            seg[x].lz += val;
            push(x, lx, rx);
            return;
        }
        int mid = (lx + rx) / 2, lf = 2 * x + 1, rt = 2 * x + 2;
        update(l, r, val, lf, lx, mid);
        update(l, r, val, rt, mid + 1, rx);
        seg[x] = merge(seg[lf], seg[rt]);
    }

    void update(int l, int r, ll val) {
        update(l, r, val, 0, 0, sz - 1);
    }

    SEG query(int l, int r, int x, int lx, int rx) {
        push(x, lx, rx);
        if (l <= lx && r >= rx)
            return seg[x];
        if (l > rx || r < lx)
            return SEG();
```

```
        int mid = (lx + rx) / 2, lf = 2 * x + 1, rt = 2 * x + 2;

        return merge(query(l, r, lf, lx, mid), query(l, r, rt, mid + 1, rx));
    }

    SEG query(int l, int r) {
        return query(l, r, 0, 0, sz - 1);
    }
};
```

2.1.3 PersistentSegmentTree

```
struct Node {
    Node *l, *r;
    ll val = 0;

    Node(int val) : l(NULL), r(NULL), val(val) {
    }

    Node() : l(NULL), r(NULL) {
    }

    Node(Node* l, Node* r) : l(l), r(r) {
        if (l != NULL) val += l->val;
        if (r != NULL) val += r->val;
    }

    void addChild() {
        l = new Node();
        r = new Node();
    }
};

struct PST {
    int n;

    PST(int n) : n(n + 1) {
    }

    Node merge(Node x, Node y) {
        Node ret;
        ret.val = x.val + y.val;
        return ret;
    }

    Node* update(Node* v, int i, int val, int lx, int rx) {
        if (lx == rx) return new Node(v->val + val);
        int mid = (lx + rx) / 2;
        if (!v->l) v->addChild();
        if (i <= mid) return new Node(update(v->l, i, val, lx, mid), v->r);
        return new Node(v->l, update(v->r, i, val, mid + 1, rx));
    }

    Node* update(Node* v, int i, int val) { return update(v, i, val, 0, n - 1); }

    Node query(Node* v, int l, int r, int lx, int rx) {
        if (l > rx || r < lx) return {};
        if (l <= lx && r >= rx) return *v;
        if (!v->l) v->addChild();
        int mid = (lx + rx) / 2;
        return merge(query(v->l, l, r, lx, mid), query(v->r, l, r, mid + 1, rx));
    }

    Node query(Node* v, int l, int r) { return query(v, l, r, 0, n - 1); }

    int getKth(Node* a, Node* b, int k, int lx, int rx) {
        if (lx == rx) return lx;
        if (!a->l) a->addChild();
        if (!b->l) b->addChild();
        int rem = b->l->val - a->l->val;
        int mid = (lx + rx) / 2;
        if (rem >= k) return getKth(a->l, b->l, k, lx, mid);
        return getKth(a->r, b->r, k - rem, mid + 1, rx);
    }

    int getKth(Node* a, Node* b, int k) { return getKth(a, b, k, 0, n - 1); }
};
```

2.1.4 PersistentSegTreeVector

```
struct Node {
    Node *l, *r;
    int val = 0;

    Node(int val): l(NULL), r(NULL), val(val) {}
    Node(): l(NULL), r(NULL) {}
    Node(Node *l, Node *r): l(l), r(r) {
        if (l != NULL) val += l->val;
        if (r != NULL) val += r->val;
    }

    void addChild() {
        l = new Node();
        r = new Node();
    }
};

struct PST {
    int n;
    PST(int n): n(n + 1) {}

    Node merge(Node x, Node y) {
        Node ret;
        ret.val = x.val + y.val;
        return ret;
    }

    Node *set(Node *v, int i, int val, int lx, int rx) {
        if (lx == rx) return new Node(val);
        int mid = (lx + rx) / 2;
        if(!v->l) v->addChild();
        if (i <= mid) return new Node(set(v->l, i, val, lx, mid), v->r);
        return new Node(v->l, set(v->r, i, val, mid + 1, rx));
    }

    Node *set(Node *v, int i, int val) { return set(v, i, val, 0, n - 1); }

    // [l, r] r is included
    Node query(Node *v, int l, int r, int lx, int rx) {
        if (l > rx || r < lx) return {};
        if (l <= lx && r >= rx) return *v;
        if(!v->l) v->addChild();
        int mid = (lx + rx) / 2;
        return merge(query(v->l, l, r, lx, mid), query(v->r, l, r, mid + 1, rx));
    }

    Node query(Node *v, int l, int r) { return query(v, l, r, 0, n - 1); }

    int getKth(Node *a, Node *b, int k, int lx, int rx) {
        if (lx == rx) return lx;
        if(!a->l) a->addChild();
        if(!b->l) b->addChild();
        int rem = b->l->val - a->l->val;
        int mid = (lx + rx) / 2;
        if (rem >= k) return getKth(a->l, b->l, k, lx, mid);
        return getKth(a->r, b->r, k - rem, mid + 1, rx);
    }

    int getKth(Node *a, Node *b, int k) { return getKth(a, b, k, 0, n - 1); }
};
```

2.1.5 SegmentTreeBeats

```
struct Node {
    int mn;
    int mnCnt;
    int secondMn;

    int mx;
    int mxCnt;
    int secondMx;

    int sum;

    int lazyAdd;
    int lazyEqual;
    bool isLazyEqual;

    Node() {
        mx = -inf;
        secondMx = -inf;
        mxCnt = 0;

        mn = inf;
        secondMn = inf;
        mnCnt = 0;

        sum = 0;

        lazyAdd = 0;
        lazyEqual = 0;
        isLazyEqual = 0;
    };

    Node(int x) {
        mx = x;
        secondMx = -inf;
        mxCnt = 1;

        mn = x;
        secondMn = inf;
        mnCnt = 1;

        sum = x;

        lazyAdd = 0;
        lazyEqual = 0;
        isLazyEqual = 0;
    }
};

struct SegTree {
    int tree_size;
    vector<Node> seg_data;

    SegTree(int n) {
        tree_size = 1;
        while (tree_size < n) tree_size *= 2;
        seg_data.resize(2 * tree_size, Node());
    }

    Node merge(Node& lf, Node& ri) {
        Node ans = Node();

        ans.sum = lf.sum + ri.sum;

        ans.mx = max(lf.mx, ri.mx);
        ans.secondMx = max(lf.secondMx, ri.secondMx);
        ans.mxCnt = 0;
        if (lf.mx == ans.mx) ans.mxCnt += lf.mxCnt;
        else ans.secondMx = max(ans.secondMx, lf.mx);
        if (ri.mx == ans.mx) ans.mxCnt += ri.mxCnt;
        else ans.secondMx = max(ans.secondMx, ri.mx);

        ans.mn = min(lf.mn, ri.mn);
        ans.secondMn = min(lf.secondMn, ri.secondMn);
```

```
        ans.mnCnt = 0;
        if (lf.mn == ans.mn) ans.mnCnt += lf.mnCnt;
        else ans.secondMn = min(ans.secondMn, lf.mn);
        if (ri.mn == ans.mn) ans.mnCnt += ri.mnCnt;
        else ans.secondMn = min(ans.secondMn, ri.mn);

        return ans;
    }

    void init(vector<int>& nums, int ni, int lx, int rx) {
        if (rx - lx == 1) {
            if (lx < nums.size())
                seg_data[ni] = Node(nums[lx]);
            return;
        }

        int mid = lx + (rx - lx) / 2;
        init(nums, 2 * ni + 1, lx, mid);
        init(nums, 2 * ni + 2, mid, rx);

        seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 * ni +
             2]);
    }

    void init(vector<int>& nums) {
        init(nums, 0, 0, tree_size);
    }

    void doPushEq(int ni, int lx, int rx, int x) {
        seg_data[ni].mn = seg_data[ni].mx = seg_data[ni].lazyEqual = x;
        seg_data[ni].isLazyEqual = 1;
        seg_data[ni].mnCnt = seg_data[ni].mxCnt = rx - lx;
        seg_data[ni].secondMx = -inf;
        seg_data[ni].secondMn = inf;

        seg_data[ni].sum = x * (rx - lx);
        seg_data[ni].lazyAdd = 0;
    }

    void doPushMinEq(int ni, int lx, int rx, int x) {
        if (seg_data[ni].mn >= x) doPushEq(ni, lx, rx, x);
        else if (seg_data[ni].mx > x) {
            if (seg_data[ni].secondMn == seg_data[ni].mx)
                seg_data[ni].secondMn = x;
            seg_data[ni].sum -= (seg_data[ni].mx - x) * seg_data[ni].
                 mxCnt;
            seg_data[ni].mx = x;
        }
    }

    void doPushMaxEq(int ni, int lx, int rx, int x) {
        if (seg_data[ni].mx <= x) doPushEq(ni, lx, rx, x);
        else if (seg_data[ni].mn < x) {
            if (seg_data[ni].secondMx == seg_data[ni].mn)
                seg_data[ni].secondMx = x;

            seg_data[ni].sum += (x - seg_data[ni].mn) * seg_data[ni].
                 mnCnt;
            seg_data[ni].mn = x;
        }
    }

    void doPushSum(int ni, int lx, int rx, int x) {
        if (seg_data[ni].mn == seg_data[ni].mx)
            doPushEq(ni, lx, rx, seg_data[ni].mn + x);
        else {
            seg_data[ni].mx += x;
            if (seg_data[ni].secondMx != -inf)
                seg_data[ni].secondMx += x;

            seg_data[ni].mn += x;
            if (seg_data[ni].secondMn != inf)
                seg_data[ni].secondMn += x;

            seg_data[ni].sum += (rx - lx) * x;
            seg_data[ni].lazyAdd += x;
        }
    }

    void propagete(int ni, int lx, int rx) {
        if (rx - lx == 1) return;

        int mid = lx + (rx - lx) / 2;
        if (seg_data[ni].isLazyEqual) {
            doPushEq(2 * ni + 1, lx, mid, seg_data[ni].lazyEqual);
            doPushEq(2 * ni + 2, mid, rx, seg_data[ni].lazyEqual);
            seg_data[ni].lazyEqual = seg_data[ni].isLazyEqual = 0;
        }
        else {
            doPushSum(2 * ni + 1, lx, mid, seg_data[ni].lazyAdd);
            doPushSum(2 * ni + 2, mid, rx, seg_data[ni].lazyAdd);
            seg_data[ni].lazyAdd = 0;

            doPushMinEq(2 * ni + 1, lx, mid, seg_data[ni].mx);
            doPushMinEq(2 * ni + 2, mid, rx, seg_data[ni].mx);

            doPushMaxEq(2 * ni + 1, lx, mid, seg_data[ni].mn);
            doPushMaxEq(2 * ni + 2, mid, rx, seg_data[ni].mn);
        }
    }

    void minimize(int l, int r, int x, int ni, int lx, int rx) {
        if (rx <= l || lx >= r || seg_data[ni].mx <= x)
            return;
        if (lx >= l && rx <= r && seg_data[ni].secondMx < x) {
            doPushMinEq(ni, lx, rx, x);
            return;
        }

        propagete(ni, lx, rx);

        int mid = lx + (rx - lx) / 2;
        minimize(l, r, x, 2 * ni + 1, lx, mid);
        minimize(l, r, x, 2 * ni + 2, mid, rx);

        seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 * ni +
             2]);
    }

    void minimize(int l, int r, int x) {
        minimize(l, r, x, 0, 0, tree_size);
    }

    void maximize(int l, int r, int x, int ni, int lx, int rx) {
        if (rx <= l || lx >= r || seg_data[ni].mn >= x)
            return;
        if (lx >= l && rx <= r && seg_data[ni].secondMn > x) {
            doPushMaxEq(ni, lx, rx, x);
            return;
        }

        propagete(ni, lx, rx);

        int mid = lx + (rx - lx) / 2;
        maximize(l, r, x, 2 * ni + 1, lx, mid);
        maximize(l, r, x, 2 * ni + 2, mid, rx);

        seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 * ni +
             2]);
    }

    void maximize(int l, int r, int x) {
        maximize(l, r, x, 0, 0, tree_size);
    }

    void add(int l, int r, int x, int ni, int lx, int rx) {
        if (rx <= l || lx >= r)
            return;
        if (lx >= l && rx <= r) {
            doPushSum(ni, lx, rx, x);
            return;
        }

        propagete(ni, lx, rx);

        int mid = lx + (rx - lx) / 2;
        add(l, r, x, 2 * ni + 1, lx, mid);
        add(l, r, x, 2 * ni + 2, mid, rx);

        seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 * ni +
             2]);
    }

    void add(int l, int r, int x) {
        add(l, r, x, 0, 0, tree_size);
    }

    void set(int l, int r, int x, int ni, int lx, int rx) {
        if (rx <= l || lx >= r)
            return;
        if (lx >= l && rx <= r) {
            doPushEq(ni, lx, rx, x);
            return;
        }

        propagete(ni, lx, rx);

        int mid = lx + (rx - lx) / 2;
        set(l, r, x, 2 * ni + 1, lx, mid);
        set(l, r, x, 2 * ni + 2, mid, rx);

        seg_data[ni] = merge(seg_data[2 * ni + 1], seg_data[2 * ni +
             2]);
    }

    void set(int l, int r, int x) {
        set(l, r, x, 0, 0, tree_size);
    }

    Node get_range(int l, int r, int ni, int lx, int rx) {
        propagete(ni, lx, rx);

        if (lx >= l && rx <= r)
            return seg_data[ni];
        if (rx <= l || lx >= r)
            return Node();

        int mid = (lx + rx) / 2;

        Node lf = get_range(l, r, 2 * ni + 1, lx, mid);
        Node ri = get_range(l, r, 2 * ni + 2, mid, rx);
        return merge(lf, ri);
    }

    int get_sum(int l, int r) {
        return get_range(l, r, 0, 0, tree_size).sum;
    }

    int get_mx(int l, int r) {
        return get_range(l, r, 0, 0, tree_size).mx;
    }

    int get_mn(int l, int r) {
        return get_range(l, r, 0, 0, tree_size).mn;
    }
};
```

2.1.6 DynamicSegTree

```
// Dynamic (Implicit) Segment Tree - Space: O(Q log N) per query
// Allocates nodes on demand for ranges up to 1e9 or 1e18
template<typename T = ll>
struct DynamicSegTree {
    struct Node {
        T val = 0;
        int lc = -1, rc = -1;
    };

```

```
    vector<Node> tree;
    ll L, R;

    DynamicSegTree(ll L = 0, ll R = 1e9) : L(L), R(R) {
        add_node();
    }

    int add_node() {
        tree.pb(Node());
        return tree.size() - 1;
    }

    void update(int node, ll l, ll r, ll pos, T val) {
        tree[node].val += val;
        if (l == r) return;
        ll mid = l + (r - l) / 2;
        if (pos <= mid) {
            if (tree[node].lc == -1) {
                int nxt = add_node();
                tree[node].lc = nxt;
            }
            update(tree[node].lc, l, mid, pos, val);
        } else {
            if (tree[node].rc == -1) {
                int nxt = add_node();
                tree[node].rc = nxt;
            }
            update(tree[node].rc, mid + 1, r, pos, val);
        }
    }

    void update(ll pos, T val) {
        update(0, L, R, pos, val);
    }

    T query(int node, ll l, ll r, ll ql, ll qr) {
        if (node == -1 || ql > r || qr < l) return 0;
        if (ql <= l && r <= qr) return tree[node].val;
        ll mid = l + (r - l) / 2;
        return query(tree[node].lc, l, mid, ql, qr) + query(tree[node].
            rc, mid + 1, r, ql, qr);
    }

    T query(ll ql, ll qr) {
        return query(0, L, R, ql, qr);
    }
};
```

2.1.7 MergeSortTree

```
// Merge Sort Tree - Space: O(N log N), Build: O(N log N), Query: O(log
    ^2 N)
// Stores sorted elements in each node to answer 2D range count queries
template<typename T = int>
struct MergeSortTree {
    int n;
    vector<vector<T>> tree;

    MergeSortTree(const vector<T>& a) : n(a.size()), tree(4 * n) {
        build(1, 0, n - 1, a);
    }

    void build(int node, int l, int r, const vector<T>& a) {
        if (l == r) {
            tree[node] = {a[l]};
            return;
        }
        int mid = (l + r) / 2;
        build(2 * node, l, mid, a);
        build(2 * node + 1, mid + 1, r, a);
        merge(all(tree[2 * node]), all(tree[2 * node + 1]),
            back_inserter(tree[node]));
    }

    // Returns number of elements <= k in index range [ql, qr]
    int query_count_le(int node, int l, int r, int ql, int qr, T k) {
        if (ql > r || qr < l) return 0;
        if (ql <= l && r <= qr) {
            return upper_bound(all(tree[node]), k) - tree[node].begin()
                ;
        }
        int mid = (l + r) / 2;
        return query_count_le(2 * node, l, mid, ql, qr, k) +
            query_count_le(2 * node + 1, mid + 1, r, ql, qr, k);
    }

    int query_count_le(int ql, int qr, T k) {
        return query_count_le(1, 0, n - 1, ql, qr, k);
    }
};
```

2.1.8 IterativeSegTree

```
// Iterative (Non-Recursive) Segment Tree - Space: O(N), Update: O(log
    N), Query: O(log N)
// Fast, cache-friendly implementation for point update and range query
template<typename T = ll>
struct IterativeSegTree {
    int n;
    vector<T> tree;
    T neutral;
    function<T(T, T)> combine;

    IterativeSegTree(int n = 0, T neutral = 0, function<T(T, T)>
        combine = [](T a, T b) { return a + b; })
        : n(n), tree(2 * n, neutral), neutral(neutral), combine(combine
            ) {}

    IterativeSegTree(const vector<T>& a, T neutral = 0, function<T(T, T
        )> combine = [](T a, T b) { return a + b; })
        : n(a.size()), tree(2 * a.size(), neutral), neutral(neutral),
            combine(combine) {
        for (int i = 0; i < n; i++) tree[n + i] = a[i];
        for (int i = n - 1; i > 0; i--) tree[i] = combine(tree[i << 1],
            tree[i << 1 | 1]);
    }

    void update(int p, T val) {
        for (tree[p += n] = val; p > 1; p >>= 1) {
            tree[p >> 1] = combine(tree[p], tree[p ^ 1]);
        }
    }

    // Range query on [l, r] inclusive
    T query(int l, int r) {
        T resl = neutral, resr = neutral;
        for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
            if (l & 1) resl = combine(resl, tree[l++]);
            if (r & 1) resr = combine(tree[--r], resr);
        }
        return combine(resl, resr);
    }
};
```

2.1.9 SegTree2D

```
// 2D Segment Tree - Space: O(N * M), Update: O(log N * log M), Query:
    O(log N * log M)
// Supports point updates and 2D subgrid range sum queries
template<typename T = ll>
struct SegTree2D {
    int n, m;
    vector<vector<T>> tree;

    SegTree2D(int n = 0, int m = 0) : n(n), m(m), tree(4 * n, vector<T
        >(4 * m, 0)) {}

    void update_y(int vx, int lx, int rx, int vy, int ly, int ry, int x
        , int y, T val) {
        if (ly == ry) {
            if (lx == rx) tree[vx][vy] += val;
            else tree[vx][vy] = tree[2 * vx][vy] + tree[2 * vx + 1][vy
                ];
            return;
        }
        int my = (ly + ry) / 2;
        if (y <= my) update_y(vx, lx, rx, 2 * vy, ly, my, x, y, val);
        else update_y(vx, lx, rx, 2 * vy + 1, my + 1, ry, x, y, val);
        tree[vx][vy] = tree[vx][2 * vy] + tree[vx][2 * vy + 1];
    }

    void update_x(int vx, int lx, int rx, int x, int y, T val) {
        if (lx != rx) {
            int mx = (lx + rx) / 2;
            if (x <= mx) update_x(2 * vx, lx, mx, x, y, val);
            else update_x(2 * vx + 1, mx + 1, rx, x, y, val);
        }
        update_y(vx, lx, rx, 1, 0, m - 1, x, y, val);
    }

    void add(int x, int y, T val) {
        update_x(1, 0, n - 1, x, y, val);
    }

    T query_y(int vx, int vy, int ly, int ry, int qy1, int qy2) {
        if (qy1 > ry || qy2 < ly) return 0;
        if (qy1 <= ly && ry <= qy2) return tree[vx][vy];
        int my = (ly + ry) / 2;
        return query_y(vx, 2 * vy, ly, my, qy1, qy2) + query_y(vx, 2 *
            vy + 1, my + 1, ry, qy1, qy2);
    }

    T query_x(int vx, int lx, int rx, int qx1, int qx2, int qy1, int
        qy2) {
        if (qx1 > rx || qx2 < lx) return 0;
        if (qx1 <= lx && rx <= qx2) return query_y(vx, 1, 0, m - 1, qy1
            , qy2);
        int mx = (lx + rx) / 2;
        return query_x(2 * vx, lx, mx, qx1, qx2, qy1, qy2) + query_x(2
            * vx + 1, mx + 1, rx, qx1, qx2, qy1, qy2);
    }

    T query(int x1, int y1, int x2, int y2) {
        return query_x(1, 0, n - 1, x1, x2, y1, y2);
    }
};
```

2.2 Binary Indexed Tree (BIT)

2.2.1 FenwickTree

```
// Fenwick Tree (Binary Indexed Tree) - 0-based indexing
// Space: O(N), Time: O(log N) per update/query
template<typename T = ll>
struct FenwickTree {
    int n;
    vector<T> tree;

    FenwickTree(int n = 0) : n(n), tree(n + 1, 0) {}

    void add(int i, T delta) {
        for (i++; i <= n; i += i & -i) tree[i] += delta;
    }

    void set(int i, T val) {
        add(i, val - query(i, i));
    }

    T query(int i) {
        T sum = 0;
        for (i++; i > 0; i -= i & -i) sum += tree[i];
        return sum;
    }

    T query(int l, int r) {
        if (l > r) return 0;
        return query(r) - query(l - 1);
```

```
    }

    // Returns first 0-based index where prefix sum >= val
    int lower_bound(T val) {
        int pos = 0;
        for (int sz = 1 << (n ? __lg(n) : 0); sz > 0; sz >>= 1) {
            if (pos + sz <= n && tree[pos + sz] < val) {
                val -= tree[pos + sz];
                pos += sz;
            }
        }
        return pos;
    }
};
```

2.2.2 FenwickTree2D

```
// 2D Fenwick Tree - 0-based indexing
// Space: O(N * M), Time: O(log N * log M) per update/query
template<typename T = ll>
struct FenwickTree2D {
    int n, m;
    vector<vector<T>> tree;

    FenwickTree2D(int n = 0, int m = 0) : n(n), m(m), tree(n + 1,
         vector<T>(m + 1, 0)) {}

    void add(int x, int y, T val) {
        for (int i = x + 1; i <= n; i += i & -i) {
            for (int j = y + 1; j <= m; j += j & -j) {
                tree[i][j] += val;
            }
        }
    }

    void set(int x, int y, T val) {
        add(x, y, val - query(x, y, x, y));
    }

    T query(int x, int y) {
        T sum = 0;
        for (int i = min(x + 1, n); i > 0; i -= i & -i) {
            for (int j = min(y + 1, m); j > 0; j -= j & -j) {
                sum += tree[i][j];
            }
        }
        return sum;
    }

    // 2D Subgrid Range Sum from (x1, y1) to (x2, y2) inclusive
    T query(int x1, int y1, int x2, int y2) {
        if (x1 > x2 || y1 > y2) return 0;
        return query(x2, y2) - query(x1 - 1, y2) - query(x2, y1 - 1) +
             query(x1 - 1, y1 - 1);
    }
};
```

2.2.3 Offline2DFenwickTree

```
// Offline 2D Fenwick Tree (Compressed 2D BIT)
// Space: O(N + Q log N), Time: O(log^2 N) per query
template<typename T = ll>
struct BIT2D {
    int n;
    vector<vector<int>> vals;
    vector<vector<T>> bit;

    int ind(const vector<int>& v, int x) {
        return upper_bound(all(v), x) - v.begin() - 1;
    }

    // n: max row index, todo: all (r, c) update coordinates that will
         be called
    BIT2D(int n, vector<array<int, 2>> todo) : n(n + 1), vals(n + 1),
         bit(n + 1) {
        sort(all(todo), [](const auto& a, const auto& b) { return a[1]
             < b[1]; });

        for (int i = 0; i <= n; i++) vals[i].pb(INT_MIN);
        for (auto [r, c] : todo) {
            for (int x = r; x <= n; x |= x + 1) {
                if (vals[x].back() != c) vals[x].pb(c);
            }
        }
        for (int i = 0; i <= n; i++) bit[i].resize(vals[i].size(), 0);
    }

    void add(int r, int c, T val) {
        for (; r <= n; r |= r + 1) {
            for (int i = ind(vals[r], c); i < (int)bit[r].size(); i |=
                 i + 1) {
                bit[r][i] += val;
            }
        }
    }

    T query(int r, int c) {
        T res = 0;
        for (; r >= 0; r = (r & (r + 1)) - 1) {
            for (int i = ind(vals[r], c); i >= 0; i = (i & (i + 1)) -
                 1) {
                res += bit[r][i];
            }
        }
        return res;
    }

    T query(int r1, int c1, int r2, int c2) {
        return query(r2, c2) - query(r2, c1 - 1) - query(r1 - 1, c2) +
             query(r1 - 1, c1 - 1);
    }

    void reset() {
        for (auto& v : bit) fill(all(v), 0);
    }
};
```

2.3 Sparse Table

2.3.1 SparseTable

```
// Sparse Table (Range Minimum/Maximum Query) - O(N log N) build, O(1)
     query
template<typename T, typename F = function<T(T, T)>>
struct SparseTable {
    int n;
    vector<vector<T>> mat;
    F func;

    SparseTable() = default;
    SparseTable(const vector<T>& a, F f = [](T x, T y) { return max(x,
         y); }) : n(a.size()), func(f) {
        int k = n ? __lg(n) + 1 : 0;
        mat.assign(k, a);
        for (int j = 1; j < k; j++) {
            for (int i = 0; i + (1 << j) <= n; i++) {
                mat[j][i] = func(mat[j - 1][i], mat[j - 1][i + (1 << (j
                     - 1))]);
            }
        }
    }

    T query(int l, int r) const {
        int j = __lg(r - l + 1);
        return func(mat[j][l], mat[j][r - (1 << j) + 1]);
    }
};
```

2.3.2 SparseTable2D

```
// 2D Sparse Table - O(N * M * log N * log M) build, O(1) query
template<typename T = ll>
struct SparseTable2D {
    int n, m;
    // jmp[kr][kc][i][j] = max in 2^kr x 2^kc subgrid starting at (i, j
         )
    vector<vector<vector<vector<T>>>> jmp;

    SparseTable2D(const vector<vector<T>>& grid) {
        n = grid.size();
        if (!n) return;
        m = grid[0].size();
        if (!m) return;

        int log_n = __lg(n) + 1, log_m = __lg(m) + 1;
        jmp.assign(log_n, vector<vector<vector<T>>>(log_m, vector<
             vector<T>>(n, vector<T>(m))));

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                jmp[0][0][i][j] = grid[i][j];
            }
        }

        for (int kc = 1; kc < log_m; kc++) {
            for (int i = 0; i < n; i++) {
                for (int j = 0; j + (1 << kc) <= m; j++) {
                    jmp[0][kc][i][j] = max(jmp[0][kc - 1][i][j], jmp
                         [0][kc - 1][i][j + (1 << (kc - 1))]);
                }
            }
        }

        for (int kr = 1; kr < log_n; kr++) {
            for (int kc = 0; kc < log_m; kc++) {
                for (int i = 0; i + (1 << kr) <= n; i++) {
                    for (int j = 0; j + (1 << kc) <= m; j++) {
                        jmp[kr][kc][i][j] = max(jmp[kr - 1][kc][i][j],
                             jmp[kr - 1][kc][i + (1 << (kr - 1))][j]);
                    }
                }
            }
        }
    }

    T query(int r1, int c1, int r2, int c2) const {
        int kr = __lg(r2 - r1 + 1), kc = __lg(c2 - c1 + 1);
        return max({jmp[kr][kc][r1][c1],
                    jmp[kr][kc][r1][c2 - (1 << kc) + 1],
                    jmp[kr][kc][r2 - (1 << kr) + 1][c1],
                    jmp[kr][kc][r2 - (1 << kr) + 1][c2 - (1 << kc) +
                         1]});
    }
};
```

2.3.3 SquareSparseTable

```
// 2D Square Sparse Table - O(N * M * log(min(N, M))) build, O(1)
     square query
template<typename T = ll>
struct SquareSparseTable {
    int n, m;
    // jmp[k][i][j] = max of 2^k x 2^k square starting at (i, j)
    vector<vector<vector<T>>> jmp;

    SquareSparseTable(const vector<vector<T>>& grid) {
        n = grid.size();
        if (!n) return;
        m = grid[0].size();
        if (!m) return;

        int log_max = __lg(min(n, m)) + 1;
        jmp.assign(log_max, vector<vector<T>>(n, vector<T>(m)));

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                jmp[0][i][j] = grid[i][j];
```

```
            }
        }

        for (int k = 1; k < log_max; k++) {
            int pw = 1 << (k - 1);
            for (int i = 0; i + (pw * 2) <= n; i++) {
                for (int j = 0; j + (pw * 2) <= m; j++) {
                    jmp[k][i][j] = max({jmp[k - 1][i][j],
                                        jmp[k - 1][i][j + pw],
                                        jmp[k - 1][i + pw][j],
                                        jmp[k - 1][i + pw][j + pw]});
                }
            }
        }
    }

    // Queries max in square subgrid from (r1, c1) to (r2, c2)
    T query(int r1, int c1, int r2, int c2) const {
        int k = __lg(r2 - r1 + 1), pw = 1 << k;
        return max({jmp[k][r1][c1],
                    jmp[k][r1][c2 - pw + 1],
                    jmp[k][r2 - pw + 1][c1],
                    jmp[k][r2 - pw + 1][c2 - pw + 1]});
    }
};
```

2.4 Trie

2.4.1 StringTrie

```
// Prefix Trie for Strings
// Time: O(L) per insert/query, Space: O(N * L * Alphabet)
struct Trie {
    vector<vector<int>> nxt;
    vector<int> cnt, end_cnt;
    int alpha;

    Trie(int alpha = 26) : alpha(alpha) {
        add_node();
    }

    int add_node() {
        nxt.emplace_back(alpha, -1);
        cnt.emplace_back(0);
        end_cnt.emplace_back(0);
        return nxt.size() - 1;
    }

    void insert(const string& s, int val = 1) {
        int u = 0;
        cnt[u] += val;
        for (char c : s) {
            int ch = c - 'a';
            if (nxt[u][ch] == -1) nxt[u][ch] = add_node();
            u = nxt[u][ch];
            cnt[u] += val;
        }
        end_cnt[u] += val;
    }

    int count_prefix(const string& s) {
        int u = 0;
        for (char c : s) {
            int ch = c - 'a';
            if (nxt[u][ch] == -1) return 0;
            u = nxt[u][ch];
        }
        return cnt[u];
    }
};
```

2.4.2 BinaryTrie

```
// Binary Trie for 64-bit integers and XOR operations
// Time: O(BITS) per operation
struct BinaryTrie {
    vector<array<int, 2>> nxt;
    vector<int> cnt;
    int max_bit;

    BinaryTrie(int max_bit = 60) : max_bit(max_bit) {
        add_node();
    }

    int add_node() {
        nxt.push_back({-1, -1});
        cnt.push_back(0);
        return nxt.size() - 1;
    }

    void insert(ll x, int val = 1) {
        int u = 0;
        cnt[u] += val;
        for (int i = max_bit; i >= 0; i--) {
            int b = (x >> i) & 1;
            if (nxt[u][b] == -1) nxt[u][b] = add_node();
            u = nxt[u][b];
            cnt[u] += val;
        }
    }

    // Returns element in trie maximizing (x ^ element)
    ll max_xor(ll x) const {
        if (!cnt[0]) return -1e18;
        int u = 0;
        ll res = 0;
        for (int i = max_bit; i >= 0; i--) {
            int b = (x >> i) & 1;
            int want = b ^ 1;
            if (nxt[u][want] != -1 && cnt[nxt[u][want]] > 0) {
                res |= (1LL << i);
                u = nxt[u][want];
            } else {
                u = nxt[u][b];
            }
        }
        return res;
    }
};
```

2.4.3 PersistentTrie

```
// Persistent Binary Trie - Space: O(N * BITS), Time: O(BITS) per insert/query
// Supports versioning and range max XOR queries between versions [l_ver, r_ver]
struct PersistentTrie {
    struct Node {
        int cnt = 0;
        array<int, 2> nxt = {-1, -1};
    };

    vector<Node> tree;
    vector<int> roots;
    int max_bit;

    PersistentTrie(int max_bit = 30) : max_bit(max_bit) {
        roots.pb(add_node());
    }

    int add_node(int old_node = -1) {
        if (old_node != -1) {
            tree.pb(tree[old_node]);
        } else {
            tree.pb(Node());
        }
        return tree.size() - 1;
    }

    // Inserts x starting from prev_root and returns the new version root
    int insert(int prev_root, ll x) {
        int new_root = add_node(prev_root);
        int u = new_root, prev_u = prev_root;
        tree[u].cnt++;

        for (int i = max_bit; i >= 0; i--) {
            int b = (x >> i) & 1;
            int prev_child = (prev_u != -1) ? tree[prev_u].nxt[b] : -1;
            tree[u].nxt[b] = add_node(prev_child);
            u = tree[u].nxt[b];
            if (prev_u != -1) prev_u = tree[prev_u].nxt[b];
            tree[u].cnt++;
        }
        roots.pb(new_root);
        return new_root;
    }

    // Returns max (x ^ val) over elements inserted in version range [l_ver, r_ver]
    ll max_xor(int l_ver, int r_ver, ll x) const {
        int u_r = roots[r_ver], u_l = (l_ver > 0) ? roots[l_ver - 1] : -1;
        ll res = 0;

        for (int i = max_bit; i >= 0; i--) {
            int b = (x >> i) & 1;
            int want = b ^ 1;

            int count_r = (u_r != -1 && tree[u_r].nxt[want] != -1) ? tree[tree[u_r].nxt[want]].cnt : 0;
            int count_l = (u_l != -1 && tree[u_l].nxt[want] != -1) ? tree[tree[u_l].nxt[want]].cnt : 0;

            if (count_r - count_l > 0) {
                res |= (1LL << i);
                u_r = tree[u_r].nxt[want];
                u_l = (u_l != -1 && tree[u_l].nxt[want] != -1) ? tree[u_l].nxt[want] : -1;
            } else {
                u_r = (u_r != -1 && tree[u_r].nxt[b] != -1) ? tree[u_r].nxt[b] : -1;
                u_l = (u_l != -1 && tree[u_l].nxt[b] != -1) ? tree[u_l].nxt[b] : -1;
            }
        }
        return res;
    }
};
```

2.5 Disjoint Set Union (DSU)

2.5.1 DSU

```
// Disjoint Set Union (DSU) - Path Compression & Union by Size
// Time: O(alpha(N)) per operation
struct DSU {
    vector<int> par, sz;

    DSU(int n = 0) : par(n), sz(n, 1) {
        iota(all(par), 0);
    }

    int find(int x) {
        return par[x] == x ? x : par[x] = find(par[x]);
    }

    bool same(int x, int y) {
        return find(x) == find(y);
    }

    bool join(int x, int y) {
        x = find(x);
        y = find(y);
        if (x == y) return false;
        if (sz[x] < sz[y]) swap(x, y);
        sz[x] += sz[y];
        par[y] = x;
```

```
        return true;
    }

    int size(int x) {
        return sz[find(x)];
    }
};
```

2.5.2 DSURollback

```
// DSU with Rollback - No path compression to allow backtracking
// Time: O(log N) per operation, O(1) rollback
struct DSURollback {
    vector<int> par, sz;
    vector<pair<int, int>> history; // {child_node, old_parent_size}

    DSURollback(int n = 0) : par(n), sz(n, 1) {
        iota(all(par), 0);
    }

    int find(int x) {
        return par[x] == x ? x : find(par[x]);
    }

    bool same(int x, int y) {
        return find(x) == find(y);
    }

    bool join(int x, int y) {
        x = find(x);
        y = find(y);
        if (x == y) {
            history.eb(-1, -1);
            return false;
        }
        if (sz[x] < sz[y]) swap(x, y);
        history.eb(y, sz[x]);
        sz[x] += sz[y];
        par[y] = x;
        return true;
    }

    int size(int x) {
        return sz[find(x)];
    }

    void rollback() {
        if (history.empty()) return;
        auto [y, old_sz_x] = history.back();
        history.pop_back();
        if (y != -1) {
            sz[par[y]] = old_sz_x;
            par[y] = y;
        }
    }

    int get_state() const {
        return history.size();
    }

    void rollback_to(int state) {
        while ((int)history.size() > state) rollback();
    }
};
```

2.6 Square Root Decomposition

2.6.1 MoAlgorithm

```
class MoArray {
private:
    struct Query {
        int l, r, idx;
        int bnd;

        bool operator<(const Query& other) const {
            if (bnd != other.bnd)
                return bnd < other.bnd;
            return (bnd & 1) ? (r < other.r) : (r > other.r);
        }
    };

    int n;
    int block_size;
    vector<int> a;

    // --- PROBLEM SPECIFIC STATE ---
    long long ans;
    vector<int> freq;

    inline void add(int idx) {
        if (freq[a[idx]] == 0) ans++;
        freq[a[idx]]++;
    }

    inline void remove(int idx) {
        freq[a[idx]]--;
        if (freq[a[idx]] == 0) ans--;
    }

    inline long long get_answer() const {
        return ans;
    }
    // ------------------------------

public:
    MoArray(const vector<int>& arr) : n(arr.size()), a(arr) {
        int max_val = 0;
        for (int x : a) {
            max_val = max(max_val, x);
        }
        freq.assign(max_val + 1, 0);
        ans = 0;
    }

    vector<long long> solve(const vector<pair<int, int>>& raw_queries) {
        int q = raw_queries.size();
        if (q == 0) return {};

        block_size = max(1, (int)(n / sqrt(q)));
        vector<Query> queries(q);

        for (int i = 0; i < q; ++i) {
            queries[i].l = raw_queries[i].first;
            queries[i].r = raw_queries[i].second;
            queries[i].idx = i;
            queries[i].bnd = queries[i].l / block_size;
        }

        sort(queries.begin(), queries.end());

        vector<long long> answers(q);
        int cur_l = 0, cur_r = -1;

        for (const Query& qry : queries) {
            while (cur_l > qry.l) add(--cur_l);
            while (cur_r < qry.r) add(++cur_r);
            while (cur_l < qry.l) remove(cur_l++);
            while (cur_r > qry.r) remove(cur_r--);
            answers[qry.idx] = get_answer();
        }

        return answers;
    }
};
```

2.6.2 MoOnSubtrees

```
class MoSubtree {
private:
    struct Query {
        int l, r, idx;
        int bnd;

        bool operator<(const Query& other) const {
            if (bnd != other.bnd) return bnd < other.bnd;
            return (bnd & 1) ? (r < other.r) : (r > other.r);
        }
    };

    int n;
    int block_size;
    vector<vector<int>> adj;
    vector<int> in, out, flat, clr;

    // --- PROBLEM SPECIFIC STATE ---
    int mx_freq;
    vector<int> freq;
    vector<long long> sum_freq;

    inline void add(int node_idx) {
        int c = clr[node_idx];
        sum_freq[freq[c]] -= c;
        freq[c]++;
        sum_freq[freq[c]] += c;
        if (freq[c] > mx_freq) mx_freq = freq[c];
    }

    inline void remove(int node_idx) {
        int c = clr[node_idx];
        sum_freq[freq[c]] -= c;
        freq[c]--;
        sum_freq[freq[c]] += c;
        while (mx_freq > 0 && sum_freq[mx_freq] == 0) mx_freq--;
    }

    inline long long get_answer() const {
        return sum_freq[mx_freq];
    }
    // ------------------------------

    void dfs(int u, int p) {
        in[u] = flat.size();
        flat.push_back(u);
        for (int v : adj[u]) {
            if (v == p) continue;
            dfs(v, u);
        }
        out[u] = flat.size() - 1;
    }

public:
    MoSubtree(int n, int max_val, const vector<int>& colors)
        : n(n), adj(n + 1), in(n + 1), out(n + 1), clr(colors) {
        mx_freq = 0;
        freq.assign(max_val + 1, 0);
        sum_freq.assign(n + 1, 0);
    }

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    vector<long long> solve(int root, const vector<int>& query_nodes) {
        flat.reserve(n);
        dfs(root, 0);

        int q = query_nodes.size();
        if (q == 0) return {};

        block_size = max(1, (int)(n / sqrt(q)));
        vector<Query> queries(q);

        for (int i = 0; i < q; ++i) {
            int u = query_nodes[i];
            queries[i].l = in[u];
```

```
            queries[i].r = out[u];
            queries[i].idx = i;
            queries[i].bnd = queries[i].l / block_size;
        }

        sort(queries.begin(), queries.end());

        vector<long long> answers(q);
        int cur_l = 0, cur_r = -1;

        for (const Query& qry : queries) {
            while (cur_l > qry.l) add(flat[--cur_l]);
            while (cur_r < qry.r) add(flat[++cur_r]);
            while (cur_l < qry.l) remove(flat[cur_l++]);
            while (cur_r > qry.r) remove(flat[cur_r--]);
            answers[qry.idx] = get_answer();
        }

        return answers;
    }
};
```

2.6.3 MoOnPaths

```
class MoTreePath {
private:
    int n, LOG, block_size;
    vector<vector<int>> adj, anc;
    vector<int> depth, in, out, flat;
    vector<long long> up, a;
    vector<bool> exist;

    // --- PROBLEM SPECIFIC STATE ---
    long long ans;
    vector<deque<int>> dep; // Tracks nodes by depth

    void add(int idx) {
        if (!dep[depth[idx]].empty()) {
            ans += 1ll * a[dep[depth[idx]].back()] * a[idx];
        }
        dep[depth[idx]].push_back(idx);
    }

    void remove(int idx) {
        if (dep[depth[idx]].size() == 2) {
            ans -= a[*dep[depth[idx]].begin()] * a[*next(dep[depth[idx
                ]].begin())];
        }
        if (dep[depth[idx]].front() == idx) {
            dep[depth[idx]].pop_front();
        } else {
            dep[depth[idx]].pop_back();
        }
    }

    void update(int idx) {
        int node = flat[idx];
        if (exist[node]) {
            remove(node);
        } else {
            add(node);
        }
        exist[node] = !exist[node];
    }

    inline long long get_answer() const {
        return ans;
    }
    // -------------------------------

    void dfs(int u, int p = -1) {
        if (p != -1) {
            anc[u][0] = p;
            up[u] = a[u] * a[u] + up[p];
        } else {
            depth[u] = 0;
            up[u] = a[u] * a[u];
        }
        in[u] = flat.size();
        flat.push_back(u);

        for (int v : adj[u]) {
            if (v == p) continue;
            depth[v] = depth[u] + 1;
            dfs(v, u);
        }

        out[u] = flat.size();
        flat.push_back(u); // Push again for Eulerian tour
    }

    void build_lca() {
        for (int k = 1; k < LOG; k++) {
            for (int i = 1; i <= n; i++) {
                anc[i][k] = anc[anc[i][k - 1]][k - 1];
            }
        }
    }

    int lift(int x, int k) {
        for (int bit = 0; bit < LOG; bit++) {
            if ((1 << bit) & k) x = anc[x][bit];
        }
        return x;
    }

    int lca(int u, int v) {
        if (depth[u] < depth[v]) swap(u, v);
        u = lift(u, depth[u] - depth[v]);
        if (u == v) return u;
        for (int bit = LOG - 1; bit >= 0; bit--) {
            if (anc[u][bit] != anc[v][bit]) {
                u = anc[u][bit];
                v = anc[v][bit];
            }
        }
        return anc[u][0];
    }

public:
    struct Query {
        int l, r, idx, x = -1;
        int bnd;

        bool operator<(const Query& other) const {
            if (bnd != other.bnd) return bnd < other.bnd;
            return (bnd & 1) ? (r < other.r) : (r > other.r);
        }
    };

    MoTreePath(int n_nodes, const vector<long long>& node_values)
        : n(n_nodes), a(node_values), LOG(log2(n_nodes) + 2) {

        adj.assign(n + 1, vector<int>());
        anc.assign(n + 1, vector<int>(LOG, 0));
        depth.assign(n + 1, 0);
        in.assign(n + 1, 0);
        out.assign(n + 1, 0);
        up.assign(n + 1, 0);
        exist.assign(n + 1, false);
        dep.assign(n + 1, deque<int>());
        ans = 0;
    }

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    vector<long long> solve(int root, vector<Query>& queries) {
        flat.clear();
        dfs(root);
        build_lca();

        int q = queries.size();
        if (q == 0) return {};

        block_size = max(1, (int)(flat.size() / sqrt(q)));

        for (auto& qry : queries) {
            qry.bnd = qry.l / block_size;
        }

        sort(queries.begin(), queries.end());

        vector<long long> answers(q);
        int cur_l = 0, cur_r = -1;

        for (const Query& qry : queries) {
            while (cur_l < qry.l) update(cur_l++);
            while (cur_r > qry.r) update(cur_r--);
            while (cur_l > qry.l) update(--cur_l);
            while (cur_r < qry.r) update(++cur_r);

            answers[qry.idx] = get_answer() + (qry.x != -1 ? up[qry.x]
                : 0);
        }
        return answers;
    }
};
```

2.7 Balanced Binary Search Tree (BBST)

2.7.1 Treap

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

template <typename T>
struct Treap {
    struct Node {
        T key, val, sum;
        T lazy = 0; // 1. Added lazy tag
        int sz = 1;
        uint64_t p = rng();
        Node *l = nullptr, *r = nullptr;

        Node(T k, T v = T()) : key(k), val(v), sum(v) {}
    }* root = nullptr;

    ~Treap() { destroy(root); }

    void destroy(Node* t) {
        if (!t) return;
        destroy(t->l);
        destroy(t->r);
        delete t;
    }

    int sz(Node* t) { return t ? t->sz : 0; }
    T sub_val(Node* t) { return t ? t->sum : 0; }

    // 2. The Push Function: Propagates pending updates to children
    void push(Node* t) {
        if (!t || t->lazy == 0) return;

        if (t->l) {
            t->l->key += t->lazy; // Shift the key (critical for the DP
                solution)
            t->l->val += t->lazy; // Shift the value
            t->l->sum += t->lazy * sz(t->l); // Update subtree sum
            t->l->lazy += t->lazy; // Pass the tag down
        }
        if (t->r) {
            t->r->key += t->lazy;
            t->r->val += t->lazy;
            t->r->sum += t->lazy * sz(t->r);
            t->r->lazy += t->lazy;
        }
```

```
        t->lazy = 0; // Clear the tag on the current node
    }

    Node* update(Node* t) {
        if (t) {
            t->sz = 1 + sz(t->l) + sz(t->r);
            t->sum = t->val + sub_val(t->l) + sub_val(t->r);
        }
        return t;
    }

    Node* merge(Node* l, Node* r) {
        push(l); // 3. Push before making routing decisions
        push(r);
        if (!l || !r) return l ? l : r;
        if (l->p > r->p) {
            l->r = merge(l->r, r);
            return update(l);
        }
        r->l = merge(l, r->l);
        return update(r);
    }

    pair<Node*, Node*> split(Node* t, T k) {
        if (!t) return {nullptr, nullptr};
        push(t); // 3. Push before evaluating keys for splits
        if (t->key <= k) {
            auto [l, r] = split(t->r, k);
            t->r = l;
            return {update(t), r};
        }
        auto [l, r] = split(t->l, k);
        t->l = r;
        return {l, update(t)};
    }

    // --- Core Operations ---

    // Range Update Method
    void add_range(T l_val, T r_val, T add_val) {
        if (l_val > r_val) return;

        auto [left_mid, right] = split(root, r_val);
        auto [left, mid] = split(left_mid, l_val - 1);

        // Apply the lazy tag to the entire isolated segment
        if (mid) {
            mid->key += add_val;
            mid->val += add_val;
            mid->sum += add_val * sz(mid);
            mid->lazy += add_val;
        }

        root = merge(merge(left, mid), right);
    }

    bool search(T x) {
        Node* curr = root;
        while (curr) {
            push(curr); // Push during traversal
            if (curr->key == x) return true;
            curr = (x < curr->key) ? curr->l : curr->r;
        }
        return false;
    }

    void insert(T x, T val = T()) {
        auto [l, r] = split(root, x);
        auto [l2, r2] = split(l, x - 1);
        if (!r2) {
            r2 = new Node(x, val);
        } else {
            push(r2); // Push before modifying
            r2->val += val;
            update(r2);
        }
        root = merge(merge(l2, r2), r);
    }

    void erase(T x) {
        auto [l, r] = split(root, x);
        auto [l2, r2] = split(l, x - 1);
        destroy(r2);
        root = merge(l2, r);
    }

    // --- Utility Operations ---

    Node* find_by_order(Node* t, int k) {
        if (!t) return nullptr;
        push(t); // Push before accessing children sizes
        int left_sz = sz(t->l);
        if (k < left_sz) return find_by_order(t->l, k);
        if (k == left_sz) return t;
        return find_by_order(t->r, k - left_sz - 1);
    }
    Node* find_by_order(int k) { return find_by_order(root, k); }

    int order_of_key(Node* t, T k) {
        if (!t) return 0;
        push(t); // Push before comparing keys
        if (t->key >= k) return order_of_key(t->l, k);
        return sz(t->l) + 1 + order_of_key(t->r, k);
    }
    int order_of_key(T k) { return order_of_key(root, k); }

    void print(Node* t) {
        if (!t) return;
        push(t); // Push before printing to get true values
        print(t->l);
        cout << t->key << " ";
        print(t->r);
    }
    void print() {
        print(root);
        cout << '\n';
    }
};
```

2.7.2 ImplicitTreap

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

template <typename T>
struct ImplicitTreap {
    struct Node {
        T val, sum;
        T lazy = 0;        // Lazy tag for range additions
        bool rev = false; // Lazy tag for range reversals
        int sz = 1;
        uint64_t p = rng();
        Node *l = nullptr, *r = nullptr;

        Node(T v = T()) : val(v), sum(v) {}
    }* root = nullptr;

    // Default Constructor
    ImplicitTreap() = default;

    // Vector Constructor - Builds treap in O(N) time
    ImplicitTreap(const vector<T>& a) {
        build(a);
    }

    ~ImplicitTreap() { destroy(root); }

    void destroy(Node* t) {
        if (!t) return;
        destroy(t->l);
        destroy(t->r);
        delete t;
    }

    int sz(Node* t) { return t ? t->sz : 0; }
    T sub_val(Node* t) { return t ? t->sum : 0; }

    // Propagates pending lazy updates to children
    void push(Node* t) {
        if (!t) return;

        // 1. Process pending reversal
        if (t->rev) {
            swap(t->l, t->r);
            if (t->l) t->l->rev ^= true;
            if (t->r) t->r->rev ^= true;
            t->rev = false;
        }

        // 2. Process pending value addition
        if (t->lazy != 0) {
            if (t->l) {
                t->l->val += t->lazy;
                t->l->sum += t->lazy * sz(t->l);
                t->l->lazy += t->lazy;
            }
            if (t->r) {
                t->r->val += t->lazy;
                t->r->sum += t->lazy * sz(t->r);
                t->r->lazy += t->lazy;
            }
            t->lazy = 0; // Clear the tag
        }
    }

    Node* update(Node* t) {
        if (t) {
            t->sz = 1 + sz(t->l) + sz(t->r);
            t->sum = t->val + sub_val(t->l) + sub_val(t->r);
        }
        return t;
    }

    // --- O(N) Linear Time Build ---
    void build(const vector<T>& a) {
        destroy(root);
        root = nullptr;
        if (a.empty()) return;

        vector<Node*> st;
        for (const T& val : a) {
            Node* curr = new Node(val);
            Node* last = nullptr;

            // Maintain max-heap property on random priority 'p'
            while (!st.empty() && st.back()->p < curr->p) {
                last = st.back();
                st.pop_back();
            }

            curr->l = last;
            if (!st.empty()) {
                st.back()->r = curr;
            }
            st.push_back(curr);
        }

        // Post-order pass to compute subtree sizes and sums in O(N)
        auto recompute = [&](auto& self, Node* t) -> void {
            if (!t) return;
            self(self, t->l);
            self(self, t->r);
            update(t);
        };

        root = st.front(); // Bottom of stack is the root
        recompute(recompute, root);
    }

```

```
    // --- Dynamic Array Operations ---

    Node* merge(Node* l, Node* r) {
        push(l);
        push(r);
        if (!l || !r) return l ? l : r;
        if (l->p > r->p) {
            l->r = merge(l->r, r);
            return update(l);
        }
        r->l = merge(l, r->l);
        return update(r);
    }

    pair<Node*, Node*> split(Node* t, int k) {
        if (!t) return {nullptr, nullptr};
        push(t);

        int left_sz = sz(t->l);
        if (left_sz < k) {
            auto [l, r] = split(t->r, k - left_sz - 1);
            t->r = l;
            return {update(t), r};
        } else {
            auto [l, r] = split(t->l, k);
            t->l = r;
            return {l, update(t)};
        }
    }

    int size() { return sz(root); }

    void insert(int idx, T val) {
        auto [l, r] = split(root, idx);
        Node* node = new Node(val);
        root = merge(merge(l, node), r);
    }

    void erase(int idx) {
        if (idx < 0 || idx >= sz(root)) return;
        auto [left_mid, right] = split(root, idx + 1);
        auto [left, mid] = split(left_mid, idx);
        destroy(mid);
        root = merge(left, right);
    }

    void reverse_range(int l, int r) {
        if (l >= r || l < 0 || r >= sz(root)) return;

        auto [left_mid, right] = split(root, r + 1);
        auto [left, mid] = split(left_mid, l);

        if (mid) mid->rev ^= true;

        root = merge(merge(left, mid), right);
    }

    void add_range(int l, int r, T add_val) {
        if (l > r || l < 0 || r >= sz(root)) return;

        auto [left_mid, right] = split(root, r + 1);
        auto [left, mid] = split(left_mid, l);

        if (mid) {
            mid->val += add_val;
            mid->sum += add_val * sz(mid);
            mid->lazy += add_val;
        }

        root = merge(merge(left, mid), right);
    }

    T query_range(int l, int r) {
        if (l > r || l < 0 || r >= sz(root)) return T();

        auto [left_mid, right] = split(root, r + 1);
        auto [left, mid] = split(left_mid, l);

        T ans = sub_val(mid);

        root = merge(merge(left, mid), right);
        return ans;
    }

    T get(int idx) {
        return query_range(idx, idx);
    }

    void print(Node* t) {
        if (!t) return;
        push(t);
        print(t->l);
        cout << t->val << " ";
        print(t->r);
    }

    void print() {
        print(root);
        cout << '\n';
    }
};
```

2.8 Policy Based Data Structures

2.8.1 OrderedSet

```
// Policy-Based Data Structure: ordered_set
// Time: O(log N) per operation
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace __gnu_pbds;
template<typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

// Usage:
// st.find_by_order(k): Iterator to k-th element (0-based)
// st.order_of_key(x): Number of elements strictly smaller than x
```

2.9 Monotonic Data Structures

2.9.1 MonotonicStack

```
// Monotonic Stack - Time: O(N), Space: O(N)
// Computes Next/Previous Smaller and Greater element indices for all positions
template<typename T = ll>
struct MonotonicStack {
    int n;
    vector<T> a;
    vector<int> prev_less, next_less;
    vector<int> prev_greater, next_greater;

    MonotonicStack(const vector<T>& arr) : n(arr.size()), a(arr),
        prev_less(n, -1), next_less(n, n),
        prev_greater(n, -1), next_greater(n, n) {

        vector<int> st;
        // Next Less & Previous Less
        for (int i = 0; i < n; i++) {
            while (!st.empty() && a[st.back()] >= a[i]) {
                next_less[st.back()] = i;
                st.pop_back();
            }
            if (!st.empty()) prev_less[i] = st.back();
            st.pb(i);
        }

        st.clear();
        // Next Greater & Previous Greater
        for (int i = 0; i < n; i++) {
            while (!st.empty() && a[st.back()] <= a[i]) {
                next_greater[st.back()] = i;
                st.pop_back();
            }
            if (!st.empty()) prev_greater[i] = st.back();
            st.pb(i);
        }
    }
};
```

2.9.2 TwoStackQueue

```
// Monotonic Queue via Two Stacks (Sliding Window Min/Max)
// Space: O(N), Time: O(1) amortized per push/pop/get
struct TwoStackQ {
    struct Node {
        ll mx = -llinf, mn = llinf, val;

        Node() : val(0) {}
        Node(ll x) : mx(x), mn(x), val(x) {}
    };

    stack<Node> a, b;

    int size() {
        return a.size() + b.size();
    }

    void mrg(Node &a, Node &b) {
        a.mn = min(a.mn, b.mn);
        a.mx = max(a.mx, b.mx);
    }

    // Pushes value into the queue
    void push(ll val) {
        auto nd = Node(val);
        if (!a.empty()) mrg(nd, a.top());
        a.push(nd);
    }

    // Transfers elements from stack a to stack b when b is empty
    void move() {
        while (!a.empty()) {
            auto nd = Node(a.top().val);
            if (!b.empty()) mrg(nd, b.top());
            b.push(nd);
            a.pop();
        }
    }

    // Returns min/max of all current elements in the queue in O(1)
    Node get() {
        Node res;
        if (!b.empty()) mrg(res, b.top());
        if (!a.empty()) mrg(res, a.top());
        return res;
    }

    // Removes the oldest element from the queue
    void pop() {
        if (b.empty()) move();
        if (!b.empty()) b.pop();
    }
};
```

2.9.3 MonotonicQueue2D

```
// 2D Monotonic Queue (Sliding Window 2D Min/Max) - Time: O(N * M), Space: O(N * M)
// Computes min in every K x L subgrid of an N x M matrix
template<typename T = ll>
struct MonotonicQueue2D {
    int n, m;

```

```
    // Returns a matrix res where res[i][j] is the min in subgrid [i..i
        +K-1][j..j+L-1]
    vector<vector<T>> sliding_window_2d(const vector<vector<T>>& grid,
         int K, int L) {
        n = grid.size();
        m = grid[0].size();

        // 1. Sliding window min horizontally along each row
        vector<vector<T>> row_min(n, vector<T>(m - L + 1));
        for (int i = 0; i < n; i++) {
            deque<int> dq;
            for (int j = 0; j < m; j++) {
                while (!dq.empty() && grid[i][dq.back()] >= grid[i][j])
                     dq.pop_back();
                dq.pb(j);
                if (dq.front() <= j - L) dq.pop_front();
                if (j >= L - 1) row_min[i][j - L + 1] = grid[i][dq.
                    front()];
            }
        }

        // 2. Sliding window min vertically down each column of row_min
        vector<vector<T>> res(n - K + 1, vector<T>(m - L + 1));
        for (int j = 0; j < m - L + 1; j++) {
            deque<int> dq;
            for (int i = 0; i < n; i++) {
                while (!dq.empty() && row_min[dq.back()][j] >= row_min[
                    i][j]) dq.pop_back();
                dq.pb(i);
                if (dq.front() <= i - K) dq.pop_front();
                if (i >= K - 1) res[i - K + 1][j] = row_min[dq.front()
                    ][j];
            }
        }

        return res;
    }
};
```

3 Graph Theory

3.1 Graph Traversal

3.1.1 TopoSort

```
// Topological Sort (Kahn's Algorithm) - O(V + E)
struct TopoSort {
    int n;
    vector<vector<int>> adj;
    vector<int> deg, order;

    TopoSort(int n = 0) : n(n), adj(n + 1), deg(n + 1, 0) {}

    void add_edge(int u, int v) {
        adj[u].pb(v);
        deg[v]++;
    }

    bool solve() {
        queue<int> q;
        for (int i = 1; i <= n; i++) {
            if (!deg[i]) q.push(i);
        }
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            order.pb(u);
            for (int v : adj[u]) {
                if (!--deg[v]) q.push(v);
            }
        }
        return (int)order.size() == n; // false if graph has cycles
    }
};
```

3.2 Shortest Paths

3.2.1 Dijkstra

```
// Dijkstra's Shortest Path Algorithm - O((V + E) log V)
struct Dijkstra {
    int n;
    vector<vector<pair<int, ll>>> adj;
    vector<ll> dist;
    vector<int> par;

    Dijkstra(int n = 0) : n(n), adj(n + 1), dist(n + 1, llinf), par(n +
        1, -1) {}

    void add_edge(int u, int v, ll w, bool directed = false) {
        adj[u].eb(v, w);
        if (!directed) adj[v].eb(u, w);
    }

    void solve(int src) {
        fill(all(dist), llinf);
        fill(all(par), -1);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>>
            pq;

        dist[src] = 0;
        pq.emplace(0, src);

        while (!pq.empty()) {
            auto [d, u] = pq.top();
            pq.pop();
            if (d > dist[u]) continue;

            for (auto& [v, w] : adj[u]) {
                if (dist[u] + w < dist[v]) {
                    dist[v] = dist[u] + w;
                    par[v] = u;
                    pq.emplace(dist[v], v);
                }
            }
        }
    }
};
```

3.2.2 FloydWarshall

```
// Floyd-Warshall All-Pairs Shortest Path - O(V^3)
struct FloydWarshall {
    int n;
    vector<vector<ll>> dist;

    FloydWarshall(int n = 0) : n(n), dist(n + 1, vector<ll>(n + 1,
        llinf)) {
        for (int i = 0; i <= n; i++) dist[i][i] = 0;
    }

    void add_edge(int u, int v, ll w, bool directed = false) {
        dist[u][v] = min(dist[u][v], w);
        if (!directed) dist[v][u] = min(dist[v][u], w);
    }

    void solve() {
        for (int k = 1; k <= n; k++) {
            for (int i = 1; i <= n; i++) {
                for (int j = 1; j <= n; j++) {
                    if (dist[i][k] < llinf && dist[k][j] < llinf) {
                        dist[i][j] = min(dist[i][j], dist[i][k] + dist[
                            k][j]);
                    }
                }
            }
        }
    }
};
```

3.2.3 BellmanFord

```
// Bellman-Ford Shortest Path with Negative Cycle Detection - O(V * E)
struct Edge {
    int u, v;
    ll w;
};

struct BellmanFord {
    int n;
    vector<Edge> edges;
    vector<ll> dist;
    vector<int> par;

    BellmanFord(int n = 0) : n(n), dist(n + 1, llinf), par(n + 1, -1)
        {}

    void add_edge(int u, int v, ll w) {
        edges.pb({u, v, w});
    }

    // Returns false if a negative cycle is reachable from src
    bool solve(int src) {
        fill(all(dist), llinf);
        fill(all(par), -1);
        dist[src] = 0;

        for (int i = 0; i < n - 1; i++) {
            bool relaxed = false;
            for (const auto& e : edges) {
                if (dist[e.u] < llinf && dist[e.u] + e.w < dist[e.v]) {
                    dist[e.v] = dist[e.u] + e.w;
                    par[e.v] = e.u;
                    relaxed = true;
                }
            }
            if (!relaxed) break;
        }

        for (const auto& e : edges) {
            if (dist[e.u] < llinf && dist[e.u] + e.w < dist[e.v]) {
                return false; // Negative cycle detected
            }
        }
        return true;
    }
};
```

3.3 Graph Connectivity

3.3.1 Bridges

```
// Bridges (Cut Edges) in Undirected Graph - O(V + E)
struct Bridges {
    int n, timer;
    vector<vector<int>> adj;
    vector<int> tin, low;
    vector<pair<int, int>> bridges;

    Bridges(int n = 0) : n(n), timer(0), adj(n + 1), tin(n + 1, 0), low
        (n + 1, 0) {}

    void add_edge(int u, int v) {
        adj[u].pb(v);
        adj[v].pb(u);
    }

    void dfs(int u, int p = 0) {
        tin[u] = low[u] = ++timer;
        for (int v : adj[u]) {
            if (v == p) continue;
            if (tin[v]) {
                low[u] = min(low[u], tin[v]);
            } else {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
```

```
                if (low[v] > tin[u]) bridges.eb(u, v);
            }
        }
    }

    void solve() {
        timer = 0;
        fill(all(tin), 0);
        bridges.clear();
        for (int i = 1; i <= n; i++) {
            if (!tin[i]) dfs(i);
        }
    }
};
```

3.3.2 CutPoints

```
// Articulation Points (Cut Vertices) in Undirected Graph - O(V + E)
struct ArticulationPoints {
    int n, timer;
    vector<vector<int>> adj;
    vector<int> tin, low;
    vector<bool> is_cut;

    ArticulationPoints(int n = 0) : n(n), timer(0), adj(n + 1), tin(n + 1, 0), low(n + 1, 0), is_cut(n + 1, false) {}

    void add_edge(int u, int v) {
        adj[u].pb(v);
        adj[v].pb(u);
    }

    void dfs(int u, int p = 0) {
        tin[u] = low[u] = ++timer;
        int children = 0;
        for (int v : adj[u]) {
            if (v == p) continue;
            if (tin[v]) {
                low[u] = min(low[u], tin[v]);
            } else {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] >= tin[u] && p != 0) is_cut[u] = true;
                children++;
            }
        }
        if (p == 0 && children > 1) is_cut[u] = true;
    }

    void solve() {
        timer = 0;
        fill(all(tin), 0);
        fill(all(is_cut), false);
        for (int i = 1; i <= n; i++) {
            if (!tin[i]) dfs(i);
        }
    }
};
```

3.3.3 SCC

```
// Strongly Connected Components (Tarjan's Algorithm) - O(V + E)
struct SCC {
    int n, timer = 0, num_comps = 0;
    vector<vector<int>> adj;
    vector<int> tin, low, comp;
    vector<bool> in_st;
    stack<int> st;
    vector<vector<int>> comps;

    SCC(int n = 0) : n(n), adj(n + 1), tin(n + 1, 0), low(n + 1, 0), comp(n + 1, -1), in_st(n + 1, false) {}

    void add_edge(int u, int v) {
        adj[u].pb(v);
    }

    void dfs(int u) {
        tin[u] = low[u] = ++timer;
        st.push(u);
        in_st[u] = true;

        for (int v : adj[u]) {
            if (!tin[v]) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (in_st[v]) {
                low[u] = min(low[u], tin[v]);
            }
        }

        if (low[u] == tin[u]) {
            comps.pb({});
            while (true) {
                int v = st.top();
                st.pop();
                in_st[v] = false;
                comp[v] = num_comps;
                comps.back().pb(v);
                if (u == v) break;
            }
            num_comps++;
        }
    }

    void solve() {
        for (int i = 1; i <= n; i++) {
            if (!tin[i]) dfs(i);
        }
    }

    vector<vector<int>> build_dag() {
        vector<vector<int>> dag(num_comps);
        for (int u = 1; u <= n; u++) {
            for (int v : adj[u]) {
                if (comp[u] != comp[v]) dag[comp[u]].pb(comp[v]);
            }
        }
        return dag;
    }
};
```

3.4 Satisfiability

3.4.1 2SAT

```
struct TwoSat {
    int N;
    vector<pair<int, int>> edges;

    TwoSat(int _N = 0) {
        init(_N);
    }

    void init(int _N = 0) {
        N = _N;
    }

    int addVar() { return N++; }

    // x or y, edges will be refined in the end
    void either(int x, int y) {
        x = max(2 * x, -1 - 2 * x);
        y = max(2 * y, -1 - 2 * y);
        edges.pb({x, y});
    }

    void implies(int x, int y) {
        either(~x, y);
    }

    void must(int x) {
        either(x, x);
    }

    void XOR(int x, int y) {
        either(x, y);
        either(~x, ~y);
    }

    vector<bool> solve(int _N = -1) {
        if (_N != -1) N = _N;
        SCC scc;
        scc.init(2 * N);
        for (auto e: edges) {
            scc.addEdge(e.st ^ 1, e.nd);
            scc.addEdge(e.nd ^ 1, e.st);
        }
        scc.gen();
        for (int i = 0; i < 2 * N; ++i) {
            if (scc.compOf[i] == scc.compOf[i ^ 1])return {};
        }
        vector<vector<int>> comps = scc.comps;
        reverse(all(comps));
        vector<int> compOf(2 * N);
        for (int i = 0; i < comps.size(); ++i) {
            for (auto e: comps[i])
                compOf[e] = i;
        }
        vector<int> tmp(comps.size());
        for (int i = 0; i < comps.size(); ++i) {
            if (!tmp[i]) {
                tmp[i] = 1;
                for (auto e: comps[i])
                    tmp[compOf[e ^ 1]] = -1;
            }
        }
        vector<bool> ans(N);
        for (int i = 0; i < N; ++i)
            ans[i] = tmp[compOf[2 * i]] == 1;
        return ans;
    }

    void atMostOne(const vector<int> &li) {
        if (li.size() <= 1) return;
        int last = ~li[1];
        for (int i = 2; i < li.size(); i++) {
            int next = addVar();
            implies(li[i], last);
            either(last, next);
            implies(li[i], next);
            last = ~next;
        }
        implies(li[0], last);
    }
};
```

3.5 Eulerian Path

3.5.1 EulerianPath

```
// Hierholzer's Algorithm for Eulerian Path / Circuit - O(V + E)
// Works for both Directed and Undirected graphs
struct EulerianPath {
    int n, m;
    vector<vector<pair<int, int>>> adj; // {neighbor, edge_id}
    vector<bool> used_edge;
    vector<int> in_deg, out_deg, path;

    EulerianPath(int n = 0, int m = 0) : n(n), m(m), adj(n + 1), used_edge(m, false), in_deg(n + 1, 0), out_deg(n + 1, 0) {}

    void add_edge(int u, int v, int edge_id, bool directed = true) {
        adj[u].pb({v, edge_id});
        out_deg[u]++;
        in_deg[v]++;
        if (!directed) {
            adj[v].pb({u, edge_id});
```

```
            out_deg[v]++;
            in_deg[u]++;
        }
    }

    // Finds Eulerian path starting at start_node. Returns false if
        graph is not Eulerian
    bool solve(int start_node) {
        vector<int> ptr(n + 1, 0);
        stack<int> st;
        st.push(start_node);

        while (!st.empty()) {
            int u = st.top();
            if (ptr[u] < adj[u].size()) {
                auto [v, id] = adj[u][ptr[u]++];
                if (!used_edge[id]) {
                    used_edge[id] = true;
                    st.push(v);
                }
            } else {
                path.pb(u);
                st.pop();
            }
        }
        reverse(all(path));

        // Verify that every edge was traversed
        for (bool used : used_edge) {
            if (!used) return false;
        }
        return true;
    }
};
```

3.6 Flow and Min Cut

3.6.1 Dinic

```
struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need flows
    };

    vector<int> lvl, ptr, q;
    vector<vector<Edge> > adj;

    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {
    }

    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, (int) adj[b].size(), c, c});
        adj[b].push_back({a, (int) adj[a].size() - 1, rcap, rcap});
    }

    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int &i = ptr[v]; i < (int) adj[v].size(); i++) {
            Edge &e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }

    ll calc(int s, int t) {
        ll flow = 0;
        q[0] = s;
        for (int L = 0; L < 31; L++)
            do {
                // 'int L=30' maybe faster for random data
                lvl = ptr = vector<int>((int) q.size());
                int qi = 0, qe = lvl[s] = 1;
                while (qi < qe && !lvl[t]) {
                    int v = q[qi++];
                    for (Edge e: adj[v])
                        if (!lvl[e.to] && e.c >> (30 - L))
                            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
                }
                while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
            } while (lvl[t]);
        return flow;
    }

    bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

3.6.2 MCMF

```
struct Edge {
    int to;
    int cost;
    int cap, flow, backEdge;
};

struct MCMF
{

    const int inf = 1000000010;
    int n;
    vector<vector<Edge>> g;

    MCMF(int _n) {
        n = _n + 1;
        g.resize(n);
    }

    void addEdge(int u, int v, int cap, int cost) {
        Edge e1 = {v, cost, cap, 0, (int) g[v].size()};
        Edge e2 = {u, -cost, 0, 0, (int) g[u].size()};
        g[u].push_back(e1);
        g[v].push_back(e2);
    }

    pair<int, int> minCostMaxFlow(int s, int t) {
        int flow = 0;
        int cost = 0;
        vector<int> state(n), from(n), from_edge(n);
        vector<int> d(n);
        deque<int> q;
        while (true) {
            for (int i = 0; i < n; i++)
                state[i] = 2, d[i] = inf, from[i] = -1;
            state[s] = 1;
            q.clear();
            q.push_back(s);
            d[s] = 0;
            while (!q.empty()) {
                int v = q.front();
                q.pop_front();
                state[v] = 0;
                for (int i = 0; i < (int) g[v].size(); i++) {
                    Edge e = g[v][i];
                    if (e.flow >= e.cap || (d[e.to] <= d[v] + e.cost))
                        continue;
                    int to = e.to;
                    d[to] = d[v] + e.cost;
                    from[to] = v;
                    from_edge[to] = i;
                    if (state[to] == 1) continue;
                    if (!state[to] || (!q.empty() && d[q.front()] > d[
                        to]))
                        q.push_front(to);
                    else q.push_back(to);
                    state[to] = 1;
                }
            }
            if (d[t] == inf) break;
            int it = t, addflow = inf;
            while (it != s) {
                addflow = min(addflow,
                              g[from[it]][from_edge[it]].cap
                              - g[from[it]][from_edge[it]].flow);
                it = from[it];
            }
            it = t;
            while (it != s) {
                g[from[it]][from_edge[it]].flow += addflow;
                g[it][g[from[it]][from_edge[it]].backEdge].flow -=
                    addflow;
                cost += g[from[it]][from_edge[it]].cost * addflow;
                it = from[it];
            }
            flow += addflow;
        }
        return {cost, flow};
    }
};
```

3.7 Matching

3.7.1 HopcroftKarp

```
struct HopcroftKarp {
    vector<int> leftMatch, rightMatch, dist, cur;
    vector<vector<int> > a;
    int n, m;

    HopcroftKarp() {}

    HopcroftKarp(int n, int m) {
        this->n = n;
        this->m = m;
        a = vector<vector<int> >(n);
        leftMatch = vector<int>(m, -1);
        rightMatch = vector<int>(n, -1);
        dist = vector<int>(n, -1);
        cur = vector<int>(n, -1);
    }

    void addEdge(int x, int y) {
        a[x].push_back(y);
    }

    int bfs() {
        int found = 0;
        queue<int> q;
        for (int i = 0; i < n; i++)
            if (rightMatch[i] < 0) dist[i] = 0, q.push(i);
            else dist[i] = -1;

        while (!q.empty()) {
            int x = q.front();
            q.pop();
            for (int i = 0; i < int(a[x].size()); i++) {
                int y = a[x][i];
                if (leftMatch[y] < 0) found = 1;
                else if (dist[leftMatch[y]] < 0)
                    dist[leftMatch[y]] = dist[x] + 1, q.push(leftMatch[
                        y]);
            }
        }

        return found;
    }

    int dfs(int x) {
        for (; cur[x] < int(a[x].size()); cur[x]++) {
            int y = a[x][cur[x]];
            if (leftMatch[y] < 0 || (dist[leftMatch[y]] == dist[x] + 1
                && dfs(leftMatch[y]))) {
                leftMatch[y] = x;
                rightMatch[x] = y;
```

```
                return 1;
            }
        }
        return 0;
    }

    int maxMatching() {
        int match = 0;
        while (bfs()) {
            for (int i = 0; i < n; i++) cur[i] = 0;
            for (int i = 0; i < n; i++)
                if (rightMatch[i] < 0) match += dfs(i);
        }
        return match;
    }
};
```

4 Number Theory

4.1 Euclidean Algorithm

4.1.1 ExtendedEuclid

```
long long extGCD(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    long long x1, y1;
    long long d = extGCD(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

long long modInverse(long long a, long long m) {
    long long x, y;
    long long g = extGCD(a, m, x, y);
    if (g != 1) return -1; // Inverse doesn't exist
    return (x % m + m) % m;
}
```

4.1.2 LinearDiophantine

```
// Linear Diophantine Equations (2 Variables & N Variables)
// 1. Solves a * x + b * y = c (Particular solution & GCD check)
// 2. Solves a_1*x_1 + a_2*x_2 + ... + a_n*x_n = c for N variables
struct LinearDiophantine {
    static ll ext_gcd(ll a, ll b, ll &x, ll &y) {
        if (b == 0) {
            x = 1; y = 0;
            return a;
        }
        ll x1, y1;
        ll d = ext_gcd(b, a % b, x1, y1);
        x = y1;
        y = x1 - y1 * (a / b);
        return d;
    }

    // Solves a * x + b * y = c. Returns false if no solution exists
    static bool solve_2var(ll a, ll b, ll c, ll &x, ll &y, ll &g) {
        if (a == 0 && b == 0) {
            if (c == 0) { x = y = g = 0; return true; }
            return false;
        }
        g = ext_gcd(abs(a), abs(b), x, y);
        if (c % g != 0) return false;
        x *= c / g;
        y *= c / g;
        if (a < 0) x = -x;
        if (b < 0) y = -y;
        return true;
    }

    // Solves a_1*x_1 + a_2*x_2 + ... + a_n*x_n = c for N variables.
    // Returns true if a solution exists and populates x vector
    static bool solve_nvar(const vector<ll>& a, ll c, vector<ll>& x) {
        int n = a.size();
        if (n == 0) return c == 0;
        x.assign(n, 0);

        vector<ll> prefix_g(n);
        prefix_g[0] = abs(a[0]);
        for (int i = 1; i < n; i++) prefix_g[i] = std::gcd(prefix_g[i -
              1], a[i]);

        if (prefix_g.back() == 0) return c == 0;
        if (c % prefix_g.back() != 0) return false;

        ll cur_c = c;
        for (int i = n - 1; i > 0; i--) {
            ll x_prev, y_curr, g;
            solve_2var(prefix_g[i - 1], a[i], cur_c, x_prev, y_curr, g)
                ;
            x[i] = y_curr;
            cur_c = prefix_g[i - 1] * x_prev;
        }
        x[0] = cur_c / a[0];
        return true;
    }
};
```

4.2 Primes and Divisors

4.2.1 Sieve

```
// Linear Sieve & Smallest Prime Factor (SPF) - Time: O(N), Space: O(N)
// Computes primes list, SPF array, primality test, and O(log N) prime
    factorization
struct Sieve {
    int n;
    vector<int> primes;
    vector<int> spf;

    Sieve(int n = 1e7) : n(n), spf(n + 1, 0) {
        for (int i = 2; i <= n; i++) {
            if (spf[i] == 0) {
                spf[i] = i;
                primes.pb(i);
            }
            for (int p : primes) {
                if (p > spf[i] || (ll)i * p > n) break;
                spf[i * p] = p;
            }
        }
    }

    bool is_prime(int x) const {
        return x >= 2 && x <= n && spf[x] == x;
    }

    // O(log N) prime factorization using precomputed SPF array
    vector<int> get_prime_factors(int x) const {
        vector<int> factors;
        while (x > 1) {
            factors.pb(spf[x]);
            x /= spf[x];
        }
        return factors;
    }
};
```

4.2.2 Factorization

```
// Trial Division Prime Factorization - Time: O(sqrt(N))
template <typename T = ll>
struct Factorization {
    static vector<T> get_prime_factors(T n) {
        vector<T> factors;
        for (T i = 2; i * i <= n; i++) {
            while (n % i == 0) {
                factors.pb(i);
                n /= i;
            }
        }
        if (n > 1) factors.pb(n);
        return factors;
    }
};
```

4.2.3 Divisors

```
// Trial Division Divisor Generation - Time: O(sqrt(N))
template <typename T = ll>
struct Divisors {
    static vector<T> get_divisors(T n) {
        vector<T> divs;
        for (T i = 1; i * i <= n; i++) {
            if (n % i == 0) {
                divs.pb(i);
                if (n / i != i) divs.pb(n / i);
            }
        }
        return divs;
    }
};
```

4.2.4 Sieve1e9

```
// Fast Bitset Sieve up to N = 10^9 - Time: ~0.3s for 10^9, Space: N /
    16 bytes (~60MB)
// Stores odd numbers only (index i represents number 2*i + 1)
template <int MAXPR = (int)1e9>
struct Sieve1e9 {
    bitset<MAXPR / 2> is_prime_odd;

    Sieve1e9(int n = MAXPR) {
        is_prime_odd.set();
        is_prime_odd[0] = 0; // 1 is not prime
        for (int i = 1; (2 * i + 1) * (2 * i + 1) <= n; i++) {
            if (is_prime_odd[i]) {
                int p = 2 * i + 1;
                for (int j = 2 * i * (i + 1); j <= n / 2; j += p) {
                    is_prime_odd[j] = 0;
                }
            }
        }
    }

    bool is_prime(int x) const {
        if (x == 2) return true;
        if (x < 2 || x % 2 == 0) return false;
        return is_prime_odd[x / 2];
    }
};
```

4.2.5 SegmentedSieve

```
// Segmented Sieve for Range [L, R] - Time: O(sqrt(R) + (R - L) log log
    R), Space: O(sqrt(R) + R - L)
// Computes all prime numbers in range [L, R] where R <= 1e12 and (R -
    L) <= 1e7
struct SegmentedSieve {
    static vector<ll> get_primes_in_range(ll L, ll R) {
        ll limit = sqrt(R);
        vector<bool> is_prime_small(limit + 1, true);
        vector<ll> primes;
        for (ll i = 2; i <= limit; i++) {
            if (is_prime_small[i]) {
                primes.pb(i);
                for (ll j = i * i; j <= limit; j += i) is_prime_small[j
                    ] = false;
            }
        }
```

```
        vector<bool> is_prime_range(R - L + 1, true);
        for (ll p : primes) {
            ll start = max(p * p, (L + p - 1) / p * p);
            for (ll j = start; j <= R; j += p) {
                is_prime_range[j - L] = false;
            }
        }
        if (L == 1) is_prime_range[0] = false;

        vector<ll> result;
        for (ll i = 0; i <= R - L; i++) {
            if (is_prime_range[i]) result.pb(L + i);
        }
        return result;
    }
};
```

4.3 Modular Arithmetic

4.3.1 CRT

```
long long extGCD(long long a, long long b, long long &x, long long &y)
    {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    long long x1, y1;
    long long d = extGCD(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

// Solves x = a_i (mod m_i). Returns {x, lcm(m_1, ..., m_k)}.
pair<long long, long long> CRT(const vector<long long>& a, const vector
    <long long>& m) {
    long long res = a[0], lcm = m[0];
    for (int i = 1; i < a.size(); i++) {
        long long x, y;
        long long g = extGCD(lcm, m[i], x, y);
        if ((a[i] - res) % g != 0) return {-1, -1}; // No solution

        long long step = m[i] / g;
        long long k = ((a[i] - res) / g) % step * (x % step) % step;
        res += k * lcm;
        lcm *= step;
        res = (res % lcm + lcm) % lcm;
    }
    return {res, lcm};
}
```

4.3.2 FloorSum

```
// Floor Sum - Time: O(log m), Space: O(log m)
// Computes sum_{i=0}^{n-1} floor((a * i + b) / m)
// Valid for 0 <= n, 0 < m, 0 <= a, 0 <= b
ll floor_sum(ll n, ll m, ll a, ll b) {
    ll ans = 0;
    if (a >= m) {
        ans += (n - 1) * n / 2 * (a / m);
        a %= m;
    }
    if (b >= m) {
        ans += n * (b / m);
        b %= m;
    }

    ll y_max = (a * n + b) / m;
    ll x_max = y_max * m - b;
    if (y_max == 0) return ans;
    ans += (n - (x_max + a - 1) / a) * y_max;
    ans += floor_sum(y_max, a, m, (a - x_max % a) % a);
    return ans;
}
```

5 Combinatorics

5.1 StarsAndBars

```
// Stars and Bars (Number of Solutions to x_1 + x_2 + ... + x_k = n) -
    O(1)
// 1. Non-negative integer solutions (x_i >= 0): C(n + k - 1, k - 1)
// 2. Positive integer solutions (x_i >= 1): C(n - 1, k - 1)
struct StarsAndBars {
    // Requires precomputed Combination C(n, k)
    static ll count_non_negative(ll n, ll k, function<ll(ll, ll)> nCr)
        {
        if (n < 0 || k <= 0) return 0;
        return nCr(n + k - 1, k - 1);
    }

    static ll count_positive(ll n, ll k, function<ll(ll, ll)> nCr) {
        if (n < k || k <= 0) return 0;
        return nCr(n - 1, k - 1);
    }
};
```

5.2 BurnsideLemma

```
// Burnside's Lemma & óPlya Enumeration Theorem - Time: O(N)
// Counts distinct colorings under symmetry group G
// Formula: Number of orbits = (1 / |G|) * sum_{g in G} (k ^
    number_of_cycles(g))
struct BurnsideLemma {
    // Calculates number of distinct necklets of length n with k colors
        under rotations
    static ll necklace(int n, int k) {
        ll total = 0;
        for (int i = 0; i < n; i++) {
            total += fast_pow(k, std::gcd(i, n));
        }
        return total / n;
    }

    // Calculates number of distinct bracelets of length n with k
        colors under rotation + reflection
    static ll bracelet(int n, int k) {
        ll total = 0;
        // Rotations
        for (int i = 0; i < n; i++) {
            total += fast_pow(k, std::gcd(i, n));
        }
        // Reflections
        if (n & 1) {
            total += (ll)n * fast_pow(k, (n + 1) / 2);
        } else {
            total += (ll)(n / 2) * fast_pow(k, n / 2) + (ll)(n / 2) *
                fast_pow(k, n / 2 + 1);
        }
        return total / (2 * n);
    }

private:
    static ll fast_pow(ll base, ll exp) {
        ll res = 1;
        while (exp > 0) {
            if (exp & 1) res *= base;
            base *= base;
            exp >>= 1;
        }
        return res;
    }
};
```

6 Math

6.1 Modular Arithmetic

6.1.1 ModularArithmetic

```
int fact[N], inv[N], invFact[N]; ///used in nCr(), nPr(), calcFact()

int modSum(ll a, ll b) {
    if (a < 0)
        a += MOD;
    if (b < 0)
        b += MOD;
    a += b;
    if (a >= MOD)
        a -= MOD;
    return a;

    a = (a % MOD + MOD) % MOD;
    b = (b % MOD + MOD) % MOD;
    return (a + b) % MOD;
}

int modProd(ll a, ll b) {
    if (a < 0)
        a += MOD;
    if (b < 0)
        b += MOD;
    a *= b;
    if (a >= MOD)
        a %= MOD;
    return a;


    a = (a % MOD + MOD) % MOD;
    b = (b % MOD + MOD) % MOD;
    return (a * b) % MOD;
}

int modPow(int a, ll b) {
    int result = 1;
    while (b) {
        if (b & 1)
            result = modProd(result, a);
        a = modProd(a, a);
        b >>= 1;
    }
    return result;
}


int modInverse(int a) {
    return modPow(a, MOD - 2);
}


int modDiv(int a, int b) {
    return modProd(a, modInverse(b));
}

int nCr(int n, int r) {
    if (r < 0 || r > n)
        return 0;
    //    return modDiv(fact[n], modProd(fact[r], fact[n - r]));
    return modProd(fact[n], modProd(invFact[r], invFact[n - r]));
}

int nPr(int n, int r) {
    if (r < 0 || r > n)
        return 0;
    //    return modDiv(fact[n], fact[n - r]);
    return modProd(fact[n], invFact[n - r]);
}

void build() {
    fact[0] = invFact[0] = 1;
    for (int i = 1; i < N; i++) {
        fact[i] = modProd(fact[i - 1], i);
        invFact[i] = modDiv(invFact[i - 1], i);
        inv[i] = modInverse(i);
```

```
    }
}
```

6.1.2 ModularIntClass

```
struct Mint {
    int v;

    Mint(long long val = 0) {
        v = int(val % MOD);
        if (v < 0) v += MOD;
    }

    Mint operator+(const Mint& o) const { return Mint(v + o.v); }
    Mint operator-(const Mint& o) const { return Mint(v - o.v); }
    Mint operator*(const Mint& o) const { return Mint(1LL * v * o.v); }
    Mint operator/(const Mint& o) const { return *this * o.inv(); }

    Mint& operator+=(const Mint& o) {
        v += o.v;
        if (v >= MOD) v -= MOD;
        return *this;
    }

    Mint& operator-=(const Mint& o) {
        v -= o.v;
        if (v < 0) v += MOD;
        return *this;
    }

    Mint& operator*=(const Mint& o) {
        v = int(1LL * v * o.v % MOD);
        return *this;
    }

    Mint& operator/=(const Mint& o) {
        v = int(1LL * v * o.inv().v % MOD);
        return *this;
    }

    Mint pow(long long p) const {
        Mint a = *this, res = 1;
        while (p > 0) {
            if (p & 1) res *= a;
            a *= a;
            p >>= 1;
        }
        return res;
    }

    Mint inv() const { return pow(MOD - 2); }

    friend ostream& operator<<(ostream& os, const Mint& m) {
        os << m.v;
        return os;
    }
};

Mint fact[N], inv[N], invFact[N];

Mint nCr(int n, int r) {
    if (r < 0 || r > n)
        return 0;
    // return fact[n] / (fact[r] * fact[n - r]);
    return fact[n] * invFact[r] * invFact[n - r];
}

Mint nPr(int n, int r) {
    if (r < 0 || r > n)
        return 0;
    // return fact[n] / fact[n - r];
    return fact[n] * invFact[n - r];
}

void build() {
    fact[0] = invFact[0] = 1;
    for (int i = 1; i < N; i++) {
        fact[i] = fact[i - 1] * i;
        invFact[i] = invFact[i - 1] / i;
        inv[i] = Mint(i).inv();
    }
}
```

6.2 Multiplicative Functions

6.2.1 EulerTotientPhi

```
int phi[N];

void calculatePhi() {
    for (int i = 0; i < N; ++i) phi[i] = i & 1 ? i : i / 2;
    for (int i = 3; i < N; i += 2)
        if (phi[i] == i)
            for (int j = i; j < N; j += i) phi[j] -= phi[j] / i;
}
```

6.2.2 MobiusFunction

```
// Mobius Function (n) Sieve - Time: O(N), Space: O(N)
// (n) = 1 if n is square-free with an even number of prime factors
// (n) = -1 if n is square-free with an odd number of prime factors
// (n) = 0 if n has a squared prime factor
struct Mobius {
    int n;
    vector<int> mu, primes;
    vector<bool> is_prime;

    Mobius(int n = 1e7) : n(n), mu(n + 1, 0), is_prime(n + 1, true) {
        is_prime[0] = is_prime[1] = false;
        mu[1] = 1;

        for (int i = 2; i <= n; i++) {
            if (is_prime[i]) {
                primes.pb(i);
                mu[i] = -1;
            }
            for (int p : primes) {
                if ((ll)i * p > n) break;
                is_prime[i * p] = false;
                if (i % p == 0) {
                    mu[i * p] = 0;
                    break;
                } else {
                    mu[i * p] = -mu[i];
                }
            }
        }
    }
};
```

6.3 Matrices

6.3.1 MatrixExponentiation

```
int modSum(ll a, ll b) {
    if (a < 0)
        a += MOD;
    if (b < 0)
        b += MOD;
    a += b;
    if (a >= MOD)
        a -= MOD;
    return a;
}

int modProd(ll a, ll b) {
    if (a < 0)
        a += MOD;
    if (b < 0)
        b += MOD;
    a *= b;
    if (a >= MOD)
        a %= MOD;
    return a;
}

typedef vector<vector<int>> matrix;

matrix operator*(const matrix& lhs, const matrix& rhs) {
    int n = lhs.size();
    int m = rhs[0].size();
    int s1 = lhs[0].size(), s2 = rhs.size();
    assert(s1 == s2);
    matrix ret(n, vector<int>(m));
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < s1; ++j)
            for (int k = 0; k < m; ++k)
                ret[i][k] = modSum(ret[i][k], modProd(lhs[i][j], rhs[j
                    ][k]));
    return ret;
}

matrix Identity(int n) {
    matrix ret(n, vector<int>(n));
    for (int i = 0; i < n; ++i) {
        ret[i][i] = 1;
    }
    return ret;
}

matrix mat_power(matrix x, ll p) {
    matrix res = Identity(x.size());
    while (p) {
        if (p & 1) res = (res * x);
        x = (x * x);
        p >>= 1;
    }
    return res;
}
```

6.3.2 FastMatrixPower

```
const int SZ = 2, mod = 1e9 + 7;

int modSum(ll a, ll b) {
    if (a < 0)
        a += mod;
    if (b < 0)
        b += mod;
    a += b;
    if (a >= mod)
        a -= mod;
    return a;
}

int modProd(ll a, ll b) {
    if (a < 0)
        a += mod;
    if (b < 0)
        b += mod;
    a *= b;
    if (a >= mod)
        a %= mod;
    return a;
}

typedef array<array<int, SZ>, SZ> matrix;

matrix operator*(const matrix &lhs, const matrix &rhs) {
    matrix ret{};
    for (int i = 0; i < SZ; ++i)
        for (int j = 0; j < SZ; ++j)
            for (int k = 0; k < SZ; ++k)
                ret[i][k] = modSum(ret[i][k], modProd(lhs[i][j], rhs[j
                    ][k]));
    return ret;
}
```

```
matrix Identity(int n) {
    matrix ret = {};
    for (int i = 0; i < n; ++i) {
        ret[i][i] = 1;
    }
    return ret;
}

matrix mat_power(matrix x, ll p) {
    matrix res = Identity(x.size());
    while (p) {
        if (p & 1) res = (res * x);
        x = (x * x);
        p >>= 1;
    }
    return res;
}
```

6.4 Linear Algebra

6.4.1 LinearXorBasis

```


// Linear XOR Basis - Vector Space over GF(2)
// Time: O(LOG_BITS) per insert/query
struct Basis {
    vector<ll> basis;
    int log_bits, sz;

    Basis(int log_bits = 60) : log_bits(log_bits), sz(0), basis(
         log_bits, 0) {}

    bool insert(ll x) {
        for (int i = log_bits - 1; i >= 0; i--) {
            if (!(x & (1LL << i))) continue;
            if (!basis[i]) {
                basis[i] = x;
                sz++;
                return true;
            }
            x ^= basis[i];
        }
        return false;
    }

    ll getMax(ll res = 0) const {
        for (int i = log_bits - 1; i >= 0; i--) {
            res = max(res, res ^ basis[i]);
        }
        return res;
    }
};
```

6.5 Convolution

6.5.1 FFT

```
#define rep(aa, bb, cc) for(int aa = bb; aa < cc;aa++)
#define sz(a) (int)a.size()
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1);  // (^ 10% faster if double)
    for (static int k = 2; k < n; k *= 2) {
        R.resize(n); rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
        rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
                // C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-
                    rolled)  /// include-line
                auto x = (double *)&rt[j+k], y = (double *)&a[i+j+k];
                            /// exclude-line
                C z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]);
                             /// exclude-line
                a[i + j + k] = a[i + j] - z;
                a[i + j] += z;
            }
}

template<int M> vi convMod(const vi &a, const vi &b) {
    if (a.empty() || b.empty()) return {};
    vi res(sz(a) + sz(b) - 1);
    int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
    vector<C> L(n), R(n), outs(n), outl(n);
    rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
    rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
    fft(L), fft(R);
    rep(i,0,n) {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
    }
    fft(outl), fft(outs);
    rep(i,0,sz(res)) {
        ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
        ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
    return res;
}
```

6.5.2 NTT

```
const int MOD = 998244353;
const int ROOT = 3; // Primitive root for 998244353

long long power(long long base, long long exp) {
    long long res = 1;
    base %= MOD;
    while (exp > 0) {
        if (exp % 2 == 1) res = (res * base) % MOD;
        base = (base * base) % MOD;
        exp /= 2;
    }
    return res;
}

long long modInverseFast(long long n) {
    return power(n, MOD - 2);
}

void ntt(vector<long long>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        long long wlen = power(ROOT, (MOD - 1) / len);
        if (invert) wlen = modInverseFast(wlen);
        for (int i = 0; i < n; i += len) {
            long long w = 1;
            for (int j = 0; j < len / 2; j++) {
                long long u = a[i + j], v = (a[i + j + len / 2] * w) %
                     MOD;
                a[i + j] = u + v < MOD ? u + v : u + v - MOD;
                a[i + j + len / 2] = u - v >= 0 ? u - v : u - v + MOD;
                w = (w * wlen) % MOD;
            }
        }
    }
    if (invert) {
        long long n_inv = modInverseFast(n);
        for (long long& x : a) x = (x * n_inv) % MOD;
    }
}
```

6.5.3 FWHT

```
// Fast Walsh-Hadamard Transform (FWHT) - Time: O(N log N)
// Computes Bitwise Convolutions: XOR, AND, OR for array of size N = 2^
    k
struct FWHT {
    enum Type { XOR, AND, OR };

    static void transform(vector<ll>& a, bool invert, Type type = XOR)
         {
        int n = a.size();
        for (int len = 1; 2 * len <= n; len <<= 1) {
            for (int i = 0; i < n; i += 2 * len) {
                for (int j = 0; j < len; j++) {
                    ll u = a[i + j], v = a[i + len + j];
                    if (type == XOR) {
                        a[i + j] = u + v;
                        a[i + len + j] = u - v;
                    } else if (type == AND) {
                        a[i + j] = invert ? u - v : u + v;
                    } else if (type == OR) {
                        a[i + len + j] = invert ? v - u : u + v;
                    }
                }
            }
        }
        if (type == XOR && invert) {
            for (ll &x : a) x /= n;
        }
    }

    // Computes C[k] = sum_{i op j = k} (A[i] * B[j]) where op in {XOR,
        AND, OR}
    static vector<ll> multiply(vector<ll> a, vector<ll> b, Type type =
         XOR) {
        int n = 1;
        while (n < max(a.size(), b.size())) n <<= 1;
        a.resize(n); b.resize(n);

        transform(a, false, type);
        transform(b, false, type);
        for (int i = 0; i < n; i++) a[i] *= b[i];
        transform(a, true, type);
        return a;
    }
};
```

6.5.4 GeneratingFunctions

```
// Formal Power Series (FPS) / Generating Functions - Time: O(N log N)
// Operations: Derivative, Integral, Polynomial Inverse, Logarithm ln(F
    (x))
template<int MOD = 998244353>
struct PolynomialFPS {
    typedef vector<ll> Poly;

    static Poly deriv(const Poly& f) {
        int n = f.size();
        if (n <= 1) return {0};
        Poly g(n - 1);
        for (int i = 1; i < n; i++) g[i - 1] = f[i] * i % MOD;
        return g;
    }

    static Poly integ(const Poly& f) {
        int n = f.size();
        Poly g(n + 1);
        for (int i = 0; i < n; i++) g[i + 1] = f[i] * modInverse(i + 1,
             MOD) % MOD;
```

```
        return g;
    }

    // Polynomial Inverse mod x^deg - O(N log N) via Newton's Method
    static Poly inv(const Poly& f, int deg) {
        if (deg == 1) return {modInverse(f[0], MOD)};
        Poly g = inv(f, (deg + 1) / 2);
        Poly f_cut(f.begin(), f.begin() + min((int)f.size(), deg));

        // g_new = g * (2 - f * g) mod x^deg
        Poly fg = multiply(f_cut, g);
        fg.resize(deg);
        for (int i = 0; i < deg; i++) {
            fg[i] = (i == 0 ? 2 : 0) - fg[i];
            if (fg[i] < 0) fg[i] += MOD;
        }
        Poly res = multiply(g, fg);
        res.resize(deg);
        return res;
    }

    // Polynomial Logarithm ln(F(x)) mod x^deg - O(N log N) (requires F[0] == 1)
    static Poly log(const Poly& f, int deg) {
        Poly g = integ(multiply(deriv(f), inv(f, deg)));
        g.resize(deg);
        return g;
    }

private:
    static Poly multiply(Poly a, Poly b) {
        // Basic NTT polynomial multiplication helper
        int n = 1;
        while (n < a.size() + b.size()) n <<= 1;
        Poly res(a.size() + b.size() - 1, 0);
        for (int i = 0; i < a.size(); i++) {
            for (int j = 0; j < b.size(); j++) {
                res[i + j] = (res[i + j] + a[i] * b[j]) % MOD;
            }
        }
        return res;
    }

    static ll modInverse(ll a, ll m) {
        ll x, y;
        ext_gcd(a, m, x, y);
        return (x % m + m) % m;
    }

    static ll ext_gcd(ll a, ll b, ll &x, ll &y) {
        if (b == 0) { x = 1; y = 0; return a; }
        ll x1, y1, d = ext_gcd(b, a % b, x1, y1);
        x = y1; y = x1 - y1 * (a / b);
        return d;
    }
};
```

7 Strings

7.1 String Matching

7.1.1 KMP

```
vector<int> compute_pi(const string &s) {
    int n = s.size();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j])
            j = pi[j - 1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}

vector<int> find_matches(const string &text, const string &pat) {
    int n = pat.length(), m = text.length();
    string s = pat + "#" + text;
    vector<int> pi = compute_pi(s), ans;
    for (int i = n + 1; i <= n + m; i++) { /* n + 1 is where the text starts */
        if (pi[i] == n) {
            ans.push_back(i - 2 * n); /* i - (n - 1) - (n + 1) */
        }
    }
    return ans;
}

void compute_automaton(string s, vector<vector<int>> &aut) {
    s += '#';
    int n = s.size();
    vector<int> pi = compute_pi(s);
    aut.assign(n, vector<int>(26));
    for (int i = 0; i < n; i++) {
        for (int c = 0; c < 26; c++) {
            if (i > 0 && 'a' + c != s[i])
                aut[i][c] = aut[pi[i - 1]][c];
            else
                aut[i][c] = i + ('a' + c == s[i]);
        }
    }
}
```

7.1.2 RabinKarp

```
vector<int> rabin_karp(string const &s, string const &t) {
    const int p = 31;
    const int m = 1e9 + 9;
    int S = s.size(), T = t.size();

    vector<ll> p_pow(max(S, T));
    p_pow[0] = 1;
    for (int i = 1; i < (int) p_pow.size(); i++)
        p_pow[i] = (p_pow[i - 1] * p) % m;

    vector<ll> h(T + 1, 0);
    for (int i = 0; i < T; i++)
        h[i + 1] = (h[i] + (t[i] - 'a' + 1) * p_pow[i]) % m;
    ll h_s = 0;
    for (int i = 0; i < S; i++)
        h_s = (h_s + (s[i] - 'a' + 1) * p_pow[i]) % m;

    vector<int> occurrences;
    for (int i = 0; i + S - 1 < T; i++) {
        ll cur_h = (h[i + S] + m - h[i]) % m;
        if (cur_h == h_s * p_pow[i] % m)
            occurrences.push_back(i);
    }
    return occurrences;
}
```

7.1.3 ZFunction

```
// z[i] is the length of the longest substring starting from s[i] which is also a prefix of s
vector<int> z_function(string s) {
    int n = s.length();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

7.2 Hashing

7.2.1 StringHashing

```
int modSum(ll a, ll b) {
    ///complexity: 1
    if (a < 0)
        a += MOD;
    if (b < 0)
        b += MOD;
    a += b;
    if (a >= MOD)
        a %= MOD;
    return a;
}

int modProd(ll a, ll b) {
    ///complexity: 1
    if (a < 0)
        a += MOD;
    if (b < 0)
        b += MOD;
    a *= b;
    if (a >= MOD)
        a %= MOD;
    return a;
}

int fastPow(int a, ll b) {
    ///complexity: log(b)
    if (b == 0) return 1;
    int temp = fastPow(a, b / 2);
    if (b % 2 == 0)
        return modProd(temp, temp);
    return modProd(modProd(a, temp), temp);
}


int modInverse(int a) {
    ///complexity: log(mod)
    return fastPow(a, MOD - 2);
}


struct Hash {
    vector<pair<int, int>> hash_value = {{0, 0}};
    const pair<int, int> p = {67, 97};
    vector<pair<int, int>> p_pow = {{1, 1}};
    vector<pair<int, int>> inv = {{1, 1}};

    void compute(string const &s) {
        for (char c: s) {
            if (c >= 'a' && c <= 'z') {
                c = c - 'a' + 1;
            } else if (c >= 'A' && c <= 'Z') {
                c = c - 'A' + 30;
            } else {
                c = c - '0' + 60;
            }
            int x = modSum(hash_value.back().st, modProd(c, p_pow.back().st));
            int a = modProd(p_pow.back().st, p.st);

            int y = modSum(hash_value.back().nd, modProd(c, p_pow.back().nd));
            int b = modProd(p_pow.back().nd, p.nd);

            hash_value.emplace_back(x, y);
            p_pow.emplace_back(a, b);
            if (inv.size() == 1) {
                inv.emplace_back(modInverse(a), modInverse(b));
```

```
            } else {
                inv.emplace_back(modProd(inv[1].st, inv.back().st),
                     modProd(inv[1].nd, inv.back().nd));
            }
        }
    }

    pair<int, int> get(int l, int r) {
        l++, r++;
        pair<int, int> ans;
        ans.st = modSum(hash_value[r].st, -hash_value[l - 1].st);
        ans.st = modProd(ans.st, inv[l].st);


        ans.nd = modSum(hash_value[r].nd, -hash_value[l - 1].nd);
        ans.nd = modProd(ans.nd, inv[l].nd);

        return ans;
    }
};
```

7.3 Aho Corasick

7.3.1 AhoCorasick

```
struct Mapper {
    int operator()(char c) const { return c - 'a'; }
};

template <int ALPHABET, typename Mapper>
struct Aho {
    struct Node {
        array<int, ALPHABET> nxt;
        int link = 0;
        int terminal_idx = -1; // -1 if not a terminal, else idx of
            string insertion
        int exit_link = 0; // Points to the nearest terminal suffix
        int depth = 0; // Size of the prefix the node represents
        Node() { nxt.fill(0); }
    };

    vector<Node> t;
    vector<int> bfs_order;
    Mapper get_idx; // Instance of the mapping function
    int word_count = 0; // Tracks the order of inserted strings

    // Constructor optionally accepts a custom mapper instance
    Aho(Mapper mapper = Mapper()) : get_idx(mapper) {
        t.emplace_back();
    }

    int insert(const string& p) {
        int u = 0;
        for (char c : p) {
            int v = get_idx(c);
            if (!t[u].nxt[v]) {
                t[u].nxt[v] = t.size();
                t.emplace_back();
                t.back().depth = t[u].depth + 1; // New node's depth is
                    parent's depth + 1
            }
            u = t[u].nxt[v];
        }

        // If this exact string hasn't been inserted before, assign it
            the current index
        if (t[u].terminal_idx == -1) {
            t[u].terminal_idx = word_count;
        }
        word_count++; // Increment for the next insertion

        return u;
    }

    void build() {
        queue<int> q;
        for (int c = 0; c < ALPHABET; ++c) {
            if (t[0].nxt[c]) {
                q.push(t[0].nxt[c]);
            }
        }

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            bfs_order.push_back(u);

            for (int c = 0; c < ALPHABET; ++c) {
                int v = t[u].nxt[c];
                if (v) {
                    t[v].link = t[t[u].link].nxt[c];

                    // Compute the exit link:
                    // If the immediate suffix link is a terminal,
                         point to it.
                    // Otherwise, copy its exit link.
                    t[v].exit_link = (t[t[v].link].terminal_idx != -1)
                         ? t[v].link : t[t[v].link].exit_link;

                    q.push(v);
                }
                else {
                    t[u].nxt[c] = t[t[u].link].nxt[c];
                }
            }
        }
    }

    int advance(int u, char c) {
        return t[u].nxt[get_idx(c)];
    }
};
```

7.4 Suffix Structures

7.4.1 SuffixArray

```
struct SuffixArray {
    string S;
    // sa is the suffix array with the empty suffix being sa[0]
    // lcp[i] holds the lcp between sa[i], sa[i - 1]
    vector<int> logs, sa, lcp, rank;
    vector<vector<int>> table;

    SuffixArray() {};

    SuffixArray(string &s, int lim = 256) {
        S = s;
        int n = s.size() + 1, k = 0, a, b;
        vector<int> c(s.begin(), s.end() + 1), tmp(n), frq(max(n, lim))
            ;
        c.back() = 0; //0 is less than any character
        sa = lcp = rank = tmp, iota(sa.begin(), sa.end(), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
            p = j, iota(tmp.begin(), tmp.end(), n - j);
            for (int i = 0; i < n; i++) {
                if (sa[i] >= j)
                    tmp[p++] = sa[i] - j;
            }

            fill(frq.begin(), frq.end(), 0);

            for (int i = 0; i < n; i++) frq[c[i]]++;
            for (int i = 1; i < lim; i++) frq[i] += frq[i - 1];
            for (int i = n; i--;) sa[--frq[c[tmp[i]]]] = tmp[i];

            swap(c, tmp), p = 1, c[sa[0]] = 0;
            for (int i = 1; i < n; i++)
                a = sa[i - 1], b = sa[i], c[b] = (tmp[a] == tmp[b] &&
                     tmp[a + j] == tmp[b + j]) ? p - 1 : p++;
        }

        for (int i = 1; i < n; i++) rank[sa[i]] = i;
        for (int i = 0, j; i < n - 1; lcp[rank[i++]] = k)
            for (k &&k--, j = sa[rank[i] - 1];
                    s[i + k] == s[j + k];
            k++);
    }

    void preLcp() {
        int n = S.size() + 1;
        logs = vector<int>(n + 5);
        for (int i = 2; i < n + 5; ++i) {
            logs[i] = logs[i / 2] + 1;
        }
        table = vector<vector<int>>(n, vector<int>(20));
        for (int i = 0; i < n; ++i) {
            table[i][0] = lcp[i];
        }

        for (int j = 1; j <= logs[n]; ++j) {
            for (int i = 0; i <= n - (1 << j); ++i) {
                table[i][j] = min(table[i][j - 1], table[i + (1 << (j -
                     1))][j - 1]);
            }
        }
    }

    int queryLcp(int i, int j) {
//          if (i == j)return (int) S.size() - i;
//          i = rank[i], j = rank[j];
        if (i == j)return (int) S.size() - sa[i];
        if (i > j)
            swap(i, j);
        i++;
        int len = logs[j - i + 1];
        return min(table[i][len], table[j - (1 << len) + 1][len]);
    }
};
```

7.4.2 SuffixAutomaton

```
struct SuffixAutomaton {
    struct Node {
        int len = 0, link = -1;
        map<char, int> next;
    };

    vector<Node> st;
    int sz = 1, last = 0;

    SuffixAutomaton(const string& s) {
        st.resize(s.length() * 2);
        st[0].len = 0;
        st[0].link = -1;
        for (char c : s) extend(c);
    }

    void extend(char c) {
        int cur = sz++;
        st[cur].len = st[last].len + 1;
        int p = last;
        while (p != -1 && !st[p].next.count(c)) {
            st[p].next[c] = cur;
            p = st[p].link;
        }
        if (p == -1) {
            st[cur].link = 0;
        } else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len) {
                st[cur].link = q;
            } else {
                int clone = sz++;
                st[clone].len = st[p].len + 1;
                st[clone].next = st[q].next;
                st[clone].link = st[q].link;
```

```
                while (p != -1 && st[p].next[c] == q) {
                    st[p].next[c] = clone;
                    p = st[p].link;
                }
                st[q].link = st[cur].link = clone;
            }
        }
        last = cur;
    }
};
```

7.5 Palindromes

7.5.1 Manacher

```
struct Manacher {
    vector<int> p;

    // Returns a vector where p[i] is the radius of the longest palindrome around center i
    // Centers are interleaved: 0 is before string, 1 is first char, 2 is between 1st & 2nd, etc.
    Manacher(string s) {
        string t = "#";
        for (char c : s) {
            t += c;
            t += "#";
        }
        int n = t.size();
        p.assign(n, 0);
        int c = 0, r = 0;
        for (int i = 0; i < n; i++) {
            int mirror = 2 * c - i;
            if (i < r) p[i] = min(r - i, p[mirror]);
            while (i - 1 - p[i] >= 0 && i + 1 + p[i] < n && t[i - 1 - p[i]] == t[i + 1 + p[i]]) {
                p[i]++;
            }
            if (i + p[i] > r) {
                c = i;
                r = i + p[i];
            }
        }
    }

    // Check if substring [l, r] (0-based) is a palindrome in O(1)
    bool is_palindrome(int l, int r) {
        int len = r - l + 1;
        int center = l + r + 1;
        return p[center] >= len;
    }
};
```

8 Dynamic Programming (DP)

8.1 DP Optimizations

8.1.1 DivideAndConquerDP

```
// Divide & Conquer DP Optimization - Time: O(K * N log N), Space: O(N)
// Applies when DP transition is dp[k][i] = min_{j < i} (dp[k-1][j] + cost(j+1, i))
// Condition: opt(i, j) <= opt(i, j + 1) (Monotonicity of optimal split points)
int a[N];
ll curCost;
int curL = 0, curR = -1, freq[N];
pair<int, ll> pre[N], cur[N];
// Modify at will :
int lo(int ind) { return 1; }
int hi(int ind) { return ind; }

void add(int val) {
    curCost += freq[val]++;
}

void remove(int val) {
    curCost -= --freq[val];
}

ll Cost(int l, int r) {
    while (curR < r)
        add(a[++curR]);
    while (curL > l)
        add(a[--curL]);
    while (curR > r)
        remove(a[curR--]);
    while (curL < l)
        remove(a[curL++]);
    return curCost;
}

ll f(int ind, int k) { return pre[k - 1].nd + Cost(k, ind); }
void store(int ind, int k, ll v) { cur[ind] = pair(k, v); }

void rec(int L, int R, int LO, int HI) {
    if (L > R) return;
    int mid = (L + R) >> 1;
    pair best(1linf, LO);
    for (int k = max(LO, lo(mid)); k <= min(HI, hi(mid)); ++k)
        best = min(best, pair(f(mid, k), k));
    store(mid, best.second, best.first);
    rec(L, mid - 1, LO, best.second);
    rec(mid + 1, R, best.second, HI);
}

void solve(int L, int R) { rec(L, R, -inf, inf); }
```

8.1.2 CHT

```
//To query the maximum value of the function for all x in [l, r], query only two points: l, r because the functions are convex.
struct Line {
    ll m, b;
    mutable function<const Line *()> succ;

    bool operator<(const Line &other) const {
        return m < other.m;
    }

    bool operator<(const ll &x) const {
        const Line *s = succ();
        if (!s)
            return 0;
        return b - s->b < (s->m - m) * x;
    }
};

// will maintain upper hull for maximum
struct HullDynamic : public multiset<Line, less<>> {
    bool bad(iterator y) {
        auto z = next(y);
        if (y == begin()) {
            if (z == end())
                return 0;
            return y->m == z->m && y->b <= z->b;
        }
        auto x = prev(y);
        if (z == end())
            return y->m == x->m && y->b <= x->b;
        return (ld) (x->b - y->b) * (z->m - y->m) >= (ld) (y->b - z->b) * (y->m - x->m);
    }

    void insert_line(ll m, ll b) {
//         for minimum
//        m *= -1;
//        b *= -1;
        auto y = insert({m, b});
        y->succ = [=] { return next(y) == end() ? 0 : &*next(y); };
        if (bad(y)) {
            erase(y);
            return;
        }
        while (next(y) != end() && bad(next(y)))
            erase(next(y));
        while (y != begin() && bad(prev(y)))
            erase(prev(y));
    }

    ll query(ll x) {
        auto l = *lower_bound(x);
//        for minimum
//        return -(l.m * x + l.b);
        return l.m * x + l.b;
    }
};
```

8.1.3 CHTRollback

```
// Rollback Convex Hull Trick - Space: O(N), Add: O(log N), Query: O(log N), Rollback: O(1)
// Supports inserting monotonic lines with history stack rollback for tree/dsu DP
// Template parameters:
// IS_MAX: Set to true for Maximum queries, false for Minimum queries.
// IS_INC: Set to true if slopes added are strictly Increasing, false for Decreasing.
template<bool IS_MAX = false, bool IS_INC = false>
struct RollbackCHT {
    struct Line {
        ll m, c;
        Line() : m(0), c(1linf) {}
        Line(ll m, ll c) : m(m), c(c) {}
        ll eval(ll x) const { return m * x + c; }
    };

    struct History {
        int pos;
        int sz;
        Line old_line;
    };

    vector<Line> st;
    vector<History> hist;
    int sz;

    RollbackCHT(int max_nodes) {
        st.resize(max_nodes);
        sz = 0;
    }

    bool redundant(Line l1, Line l2, Line l3) {
        __int128 dx1 = l1.m - l2.m;
        __int128 dc1 = l2.c - l1.c;
        __int128 dx2 = l2.m - l3.m;
        __int128 dc2 = l3.c - l2.c;
        return dc1 * dx2 >= dc2 * dx1;
    }

    void add(ll m, ll c) {
        // The magic happens here: automatically formats internal state
        ll m_int = IS_INC ? -m : m;
        ll c_int = IS_MAX ? -c : c;

        Line l_new(m_int, c_int);
        int pos = sz;
        int low = 1, high = sz - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (redundant(st[mid - 1], st[mid], l_new)) {
                pos = mid;
                high = mid - 1;
            } else {
                low = mid + 1;
            }
```

```
        }

        hist.push_back({pos, sz, st[pos]});
        st[pos] = l_new;
        sz = pos + 1;
    }

    void rollback() {
        if (hist.empty()) return;
        History h = hist.back();
        hist.pop_back();
        sz = h.sz;
        st[h.pos] = h.old_line;
    }

    ll query(ll x) {
        if (sz == 0) return IS_MAX ? -llinf : llinf;

        // The magic happens here: matches x to the internal geometry
        ll x_int = (IS_INC != IS_MAX) ? -x : x;

        int low = 0, high = sz - 2;
        int best = sz - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (st[mid].eval(x_int) <= st[mid + 1].eval(x_int)) {
                best = mid;
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }

        ll ans = st[best].eval(x_int);
        return IS_MAX ? -ans : ans;
    }
};
```

8.1.4 LiChaoTree

```
// Li Chao Tree - Space: O(N log X), Insert Line: O(log X), Query: O(log X)
// Dynamic segment tree evaluating line functions y = m * x + b for arbitrary queries x
// To get an instance : node *root = EMPTY;
typedef pair<long long, long long> line;
int start_x, end_x;
line neutral{0,-1e18};

ll sub(const line &l, ll x) {
    return l.first * x + l.second;
}

double inter(const line &l1, const line &l2) {
    return (l2.second - l1.second) / (l1.first - l2.first - .0);
}

extern struct node *const EMPTY;
struct node {
    line l;
    node *lf, *rt;
    node() : l(neutral), lf(this), rt(this) {}
    node(line l) : l(l), lf(EMPTY), rt(EMPTY) {}
};
node *const EMPTY = new node;

void insert(line l, int lx, int rx, node *&cur, ll ns = start_x, ll ne = end_x) {
    if(rx < ns || lx > ne) return;
    if(ns >= lx && ne <= rx) {
        if (cur == EMPTY) {
            cur = new node(l);
            return;
        }
        if (l.first == cur->l.first) {
            cur->l = max(cur->l, l);
            return;
        }

        auto x = inter(l, cur->l);
        if (x < ns || x > ne) {
            if (sub(l, ns) > sub(cur->l, ns)) cur->l = l;
            return;
        }
        ll mid = ns + (ne - ns) / 2;
        if (x < mid) {
            if (sub(l, ne) > sub(cur->l, ne)) swap(l, cur->l);
            insert(l, lx, rx, cur->lf, ns, mid);
        } else {
            if (sub(l, ns) > sub(cur->l, ns)) swap(l, cur->l);
            insert(l,lx, rx, cur->rt, mid + 1, ne);
        }
        return;
    }

    if (cur == EMPTY) {
        cur = new node(neutral);
    }
    ll mid = ns + (ne - ns) / 2;
    insert(l, lx, rx, cur->lf, ns, mid);
    insert(l, lx, rx, cur->rt, mid + 1, ne);
}

ll query(ll x, node *cur, ll ns = start_x, ll ne = end_x) {
    auto ret = sub(cur->l, x);
    if (x < ns || x > ne)
        return sub(neutral, x);
    if (ns == ne)
        return ret;
    ll mid = ns + (ne - ns) / 2;
    if (x <= mid)
        return max(ret, query(x, cur->lf, ns, mid));
    return max(ret, query(x, cur->rt, mid + 1, ne));
}
```

8.1.5 PersistentLiChaoTree

```
// Persistent Li Chao Tree - Space: O(N log X), Insert Line: O(log X), Query: O(log X)
// Version-persistent Li Chao Segment Tree for tree / history DP queries
const ll maxN = 1e7 + 5;

struct Line {
    ll m, c;

    Line() : m(0), c(llinf) {}

    Line(ll m, ll c) : m(m), c(c) {}
};

ll sub(ll x, Line l) {
    return x * l.m + l.c;
}

// Persistent Li Chao
struct Node {
// range I am responsible for
    Line line;
    Node *left, *right;

    Node() {
        left = right = NULL;
    }

    Node(ll m, ll c) {
        line = Line(m, c);
        left = right = NULL;
    }

    void extend(int l, int r) {
        if (left == NULL && l != r) {
            left = new Node();
            right = new Node();
        }
    }

    Node *copy(Node *node) {
        Node *newNode = new Node;
        newNode->left = node->left;
        newNode->right = node->right;
        newNode->line = node->line;
        return newNode;
    }

    Node *add(Line toAdd, int l, int r) {
        int mid = (l + r) / 2;
        Node *cur = copy(this);
        if (l == r) {
            if (sub(l, toAdd) < sub(l, cur->line))
                swap(toAdd, cur->line);
            return cur;
        }
        bool lef = sub(l, toAdd) < sub(l, cur->line);
        bool midE = sub(mid + 1, toAdd) < sub(mid + 1, cur->line);
        if (midE)
            swap(cur->line, toAdd);
        cur->extend(l, r);
        if (lef != midE) {
            cur->left = cur->left->add(toAdd, l, mid);
        } else {
            if (mid + 1 > r)
                return cur;
            cur->right = cur->right->add(toAdd, mid + 1, r);
        }
        return cur;
    }

    Node *add(Line toAdd) {
        return add(toAdd, -maxN + 1, maxN - 1);
    }

    ll query(ll x, int l, int r) {
        int mid = (l + r) / 2;
        if (l == r || left == NULL)
            return sub(x, line);
        extend(l, r);
        if (x <= mid)
            return min(sub(x, line), left->query(x, l, mid));
        else
            return min(sub(x, line), right->query(x, mid + 1, r));
    }

    ll query(ll x) {
        return query(x, -maxN + 1, maxN - 1);
    }

    void clear() {
        if (left != NULL) {
            left->clear();
            right->clear();
        }
        delete this;
    }
};

Node *tree[N];
```

8.1.6 KnuthDP

```
// Knuth's DP Optimization - Time: O(N^2), Space: O(N^2)
// Optimizes interval DP: dp[i][j] = min_{i <= k < j} (dp[i][k] + dp[k+1][j]) + cost(i, j)
// Applicable when opt[i][j-1] <= opt[i][j] <= opt[i+1][j]
struct KnuthDP {
    int n;
    vector<vector<ll>> dp;
```

```
    vector<vector<int>> opt;

    KnuthDP(int n = 0) : n(n), dp(n + 2, vector<ll>(n + 2, 0)), opt(n +
        2, vector<int>(n + 2, 0)) {}

    // User-defined cost function cost(i, j)
    ll solve(function<ll(int, int)> cost) {
        for (int i = 1; i <= n; i++) {
            opt[i][i] = i;
        }

        for (int len = 2; len <= n; len++) {
            for (int i = 1; i + len - 1 <= n; i++) {
                int j = i + len - 1;
                dp[i][j] = llinf;
                int L = opt[i][j - 1], R = opt[i + 1][j];
                for (int k = L; k <= min(R, j - 1); k++) {
                    ll cur = dp[i][k] + dp[k + 1][j] + cost(i, j);
                    if (cur < dp[i][j]) {
                        dp[i][j] = cur;
                        opt[i][j] = k;
                    }
                }
            }
        }
        return dp[1][n];
    }
};
```

8.1.7 AliensTrick

```
// Alien's Trick (WNS / Penalty Binary Search) - Time: O(N log(
    PENALTY_RANGE))
// Solves exact K-element selection / partition DP when cost function f
    (k) is strictly convex
// Adds a penalty lambda per selected item to drop the k constraint
struct AliensTrick {
    // Binary searches optimal penalty lambda to choose exactly K items
    // Returns minimum cost to choose exactly K items
    static ll solve(int n, int k, ll min_penalty, ll max_penalty,
        function<pair<ll, int>(ll)> dp_solver) {
        ll l = min_penalty, r = max_penalty, best_lambda = 0;

        while (l <= r) {
            ll mid = l + (r - l) / 2;
            auto [cost, count] = dp_solver(mid);
            if (count >= k) {
                best_lambda = mid;
                l = mid + 1; // Increase penalty if we used too many
                    items
            } else {
                r = mid - 1; // Decrease penalty if we used too few
                    items
            }
        }

        // True cost = cost_with_penalty - (k * best_lambda)
        auto [cost, count] = dp_solver(best_lambda);
        return cost - (ll)k * best_lambda;
    }
};
```

8.2 Bitmask DP

8.2.1 SOS_DP

```
// Computes the sum of all subsets for every bitmask in O(N * 2^N)
vector<long long> SOS_DP(int n_bits, const vector<long long>& a) {
    int max_mask = 1 << n_bits;
    vector<long long> dp = a; // dp[mask] initially holds exactly the
        value for mask

    for (int i = 0; i < n_bits; ++i) {
        for (int mask = 0; mask < max_mask; ++mask) {
            if (mask & (1 << i)) {
                dp[mask] += dp[mask ^ (1 << i)];
            }
        }
    }
    return dp; // dp[mask] now contains sum of all submasks of mask
}
```

9 Trees

9.1 Lowest Common Ancestor (LCA)

9.1.1 LCA

```
struct LCA {
    static const int B = 21;
    vector<array<int, 2>> tour;
    vector<array<array<int, 2>, B>> T;
    vector<int> d, in, lg;

    LCA(int n, auto &adj): d(n + 1), in(n + 1), lg(n*2 + 1) {
        tour.reserve(n *= 2);
        T.resize(n);
        dfs(0, 0, adj);
        for(int i = 0; i < n; ++i) lg[i]=__lg(i), T[i][0]=tour[i];
        for (int j = 1; j < B; ++j)
            for (int i = 0; i + (1 << j) - 1 < n; ++i)
                T[i][j] = min(T[i][j - 1], T[i+(1 << j - 1)][j - 1]);
    }

    void dfs(int u, int p, auto &adj) {
        in[u] = size(tour);
        tour.push_back({d[u], u});
        for (int v: adj[u]) {
            if (v == p) continue;
            d[v] = d[u] + 1;
            dfs(v, u, adj);
            tour.push_back({d[u], u});
        }
    }

    int get(int u, int v) {
        int l = min(in[u], in[v]), r = max(in[u], in[v]);
        int j = lg[r - l + 1];
        return min(T[l][j], T[r - (1 << j) + 1][j])[1];
    }

    int dist(int u, int v) { return d[u]+d[v]-2*d[get(u, v)]; }
};
```

9.2 DSU on Tree

9.2.1 DSUOnTree

```
class DSUOnTree {
private:
    struct Query {
        int idx, x;
    };

    int n;
    vector<vector<int>> adj;
    vector<int> sz, depth;
    vector<bool> big;
    vector<vector<Query>> queries;
    vector<int> ans;

    // ========================================
    //       PROBLEM SPECIFIC STATE VARIABLES
    // ========================================
    vector<char> c; // Example: Array of characters per node

    // Example state variables
    // int distinct_count = 0;
    // vector<int> freq;

    void add(int u, int p) {
        // --- ADD LOGIC HERE ---
        // Example: freq[c[u] - 'a']++;

        for (int v : adj[u]) {
            if (v == p || big[v]) continue; // Skip parent and heavy
                child[cite: 4]
            add(v, u);
        }
    }

    void remove(int u, int p) {
        // --- REMOVE LOGIC HERE ---
        // Example: freq[c[u] - 'a']--;

        for (int v : adj[u]) {
            if (v == p || big[v]) continue; // Skip parent and heavy
                child[cite: 4]
            remove(v, u);
        }
    }

    int get_answer(int u, int x = 0) {
        // --- QUERY ANSWER LOGIC HERE ---
        return 0;
    }
    // ========================================

    void pre(int u, int p = 0) {
        sz[u] = 1;
        depth[u] = depth[p] + 1;
        for (int v : adj[u]) {
            if (v == p) continue;
            pre(v, u);
            sz[u] += sz[v]; // Compute subtree size[cite: 4]
        }
    }

    void dfs(int u, int p = 0, bool keep = false) {
        int mx = -1, bigChild = -1;

        // Find the heavy child[cite: 4]
        for (int v : adj[u]) {
            if (v == p) continue;
            if (sz[v] > mx) {
                mx = sz[v];
                bigChild = v;
            }
        }

        // Process light children and clear them[cite: 4]
        for (int v : adj[u]) {
            if (v == p || v == bigChild) continue;
            dfs(v, u, false);
        }

        // Process heavy child and keep it[cite: 4]
        if (bigChild != -1) {
            dfs(bigChild, u, true);
            big[bigChild] = true;
        }

        // Add current node and its light children[cite: 4]
        add(u, p);

        // Answer all queries for the current node[cite: 4]
        for (const auto& q : queries[u]) {
            ans[q.idx] = get_answer(u, q.x);
        }

        // Reset the big child flag[cite: 4]
        if (bigChild != -1) {
            big[bigChild] = false;
        }

        // Clear state if this node is not meant to be kept[cite: 4]
        if (!keep) {
```

```
            remove(u, p);
        }
    }

public:
    // Initialize with 1-based indexing in mind
    DSUOnTree(int nodes, const vector<char>& chars) : n(nodes), c(chars
        ) {
        adj.assign(n + 1, vector<int>());
        sz.assign(n + 1, 0);
        depth.assign(n + 1, 0);
        big.assign(n + 1, false);
        queries.assign(n + 1, vector<Query>());
        // Initialize problem-specific arrays here (e.g., freq.assign
            (26, 0);)
    }

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void add_query(int node, int query_idx, int x = 0) {
        queries[node].push_back({query_idx, x});
    }

    vector<int> solve(int root, int num_queries) {
        ans.assign(num_queries, 0);
        pre(root, 0); // Precalculate sizes and depths[cite: 4]
        dfs(root, 0, false); // Run the sack algorithm[cite: 4]
        return ans;
    }
};
```

9.3 Heavy Light Decomposition (HLD)

9.3.1 HLD

```
class HLD {
public:
    vector<int> par, sz, head, tin, tout, who, depth;

    int dfs1(int u, vector<vector<int> > &adj) {
        for (int &v: adj[u]) {
            if (v == par[u])continue;
            depth[v] = depth[u] + 1;
            par[v] = u;
            sz[u] += dfs1(v, adj);
            if (sz[v] > sz[adj[u][0]] || adj[u][0] == par[u]) swap(v,
                adj[u][0]);
        }
        return sz[u];
    }

    void dfs2(int u, int &timer, const vector<vector<int> > &adj) {
        tin[u] = timer++;
        for (int v: adj[u]) {
            if (v == par[u])continue;
            head[v] = (timer == tin[u] + 1 ? head[u] : v);
            dfs2(v, timer, adj);
        }
        tout[u] = timer - 1;
    }

    HLD(vector<vector<int> > adj, int r = 0)
        : par(adj.size(), -1), sz(adj.size(), 1), head(adj.size(), r),
            tin(adj.size()), tout(adj.size()),
          who(adj.size()),
          depth(adj.size()) {
        dfs1(r, adj);
        int x = 0;
        dfs2(r, x, adj);
        for (int i = 0; i < adj.size(); ++i) who[tin[i]] = i;
    }

    vector<pair<int, int> > path(int u, int v) {
        vector<pair<int, int> > res;
        for (;; v = par[head[v]]) {
            if (depth[head[u]] > depth[head[v]])swap(u, v);
            if (head[u] != head[v]) {
                res.emplace_back(tin[head[v]], tin[v]);
            } else {
                if (depth[u] > depth[v])swap(u, v);
                res.emplace_back(tin[u], tin[v]);
                return res;
            }
        }
    }

    pair<int, int> subtree(int u) {
        return {tin[u], tout[u]};
    }

    int dist(int u, int v) {
        return depth[u] + depth[v] - 2 * depth[lca(u, v)];
    }

    int lca(int u, int v) {
        for (;; v = par[head[v]]) {
            if (depth[head[u]] > depth[head[v]])swap(u, v);
            if (head[u] == head[v]) {
                if (depth[u] > depth[v])swap(u, v);
                return u;
            }
        }
    }

    bool isAncestor(int u, int v) {
        return tin[u] <= tin[v] && tout[u] >= tout[v];
    }
};
```

9.4 Centroid Decomposition

9.4.1 Centroid

```
class CentroidDecomposition {
private:
    int n;
    vector<vector<int>> adj;
    vector<int> sz, par, nodes;
    vector<bool> vis;

    int dfsSz(int u, int p) {
        sz[u] = 1;
        for (int v : adj[u]) {
            if (vis[v] || v == p) continue;
            sz[u] += dfsSz(v, u);
        }
        return sz[u];
    }

    int getCentroid(int u, int p, int total_nodes) {
        for (int v : adj[u]) {
            if (vis[v] || v == p) continue;
            if (sz[v] * 2 > total_nodes) {
                return getCentroid(v, u, total_nodes);
            }
        }
        return u;
    }

    void collect(int u, int p) {
        nodes.push_back(u);
        for (int v : adj[u]) {
            if (vis[v] || v == p) continue;
            collect(v, u);
        }
    }

    void build(int u, int p) {
        int total_nodes = dfsSz(u, p);
        int centroid = getCentroid(u, p, total_nodes);

        if (p != -1) {
            par[centroid] = p;
        } else {
            par[centroid] = centroid;
        }

        vis[centroid] = true;

        // ========================================
        //       PROBLEM SPECIFIC PROCESSING
        // ========================================
        // Process paths going through the 'centroid' here
        for (int v : adj[centroid]) {
            if (vis[v]) continue;
            nodes.clear();
            collect(v, centroid);

            // e.g., Update answers using the nodes collected from this
                subtree branch
        }
        // ========================================

        // Recursively decompose the remaining components
        for (int v : adj[centroid]) {
            if (vis[v]) continue;
            build(v, centroid);
        }
    }

public:
    // Initialize with 1-based indexing
    CentroidDecomposition(int nodes_count) : n(nodes_count) {
        adj.assign(n + 1, vector<int>());
        sz.assign(n + 1, 0);
        par.assign(n + 1, 0);
        vis.assign(n + 1, false);
    }

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    // Start decomposition, usually from node 1
    void decompose(int root = 1) {
        build(root, -1);
    }

    // Returns the parent of a node in the centroid tree
    int get_centroid_parent(int u) const {
        return par[u];
    }
};
```

9.5 Tree Diameter

9.5.1 TreeDiameter

```
// Tree Diameter via 2-BFS - Time: O(V + E), Space: O(V + E)
// Computes diameter length, endpoints (u, v), and the full diameter
    path
struct TreeDiameter {
    int n;
    vector<vector<int>> adj;
    vector<int> dist, par;

    TreeDiameter(int n = 0) : n(n), adj(n + 1), dist(n + 1), par(n + 1)
        {}

    void add_edge(int u, int v) {
        adj[u].pb(v);
        adj[v].pb(u);
```

```
    }

    int bfs(int src) {
        fill(all(dist), -1);
        queue<int> q;
        q.push(src);
        dist[src] = 0;
        par[src] = -1;

        int farthest = src;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            if (dist[u] > dist[farthest]) farthest = u;

            for (int v : adj[u]) {
                if (dist[v] == -1) {
                    dist[v] = dist[u] + 1;
                    par[v] = u;
                    q.push(v);
                }
            }
        }
        return farthest;
    }

    // Returns {diameter_length, {endpoint_u, endpoint_v}}
    pair<int, pair<int, int>> get_diameter() {
        int u = bfs(1);
        int v = bfs(u);
        return {dist[v], {u, v}};
    }

    // Returns the sequence of vertices on the diameter path from u to
        v
    vector<int> get_path() {
        auto [len, endpoints] = get_diameter();
        auto [u, v] = endpoints;
        vector<int> path;
        for (int curr = v; curr != -1; curr = par[curr]) {
            path.pb(curr);
        }
        reverse(all(path));
        return path;
    }
};
```

9.6 Euler Tour

9.6.1 EulerTour

```
// Euler Tour Technique (Tree Flattening into 1D Array Intervals) - O(V
    + E)
// Subtree of node u corresponds to contiguous range [in_time[u],
    out_time[u]] in flat_tree
struct EulerTour {
    int n, timer = 0;
    vector<vector<int>> adj;
    vector<int> in_time, out_time, flat_tree;

    EulerTour(int n = 0) : n(n), adj(n + 1), in_time(n + 1), out_time(n
        + 1) {}

    void add_edge(int u, int v) {
        adj[u].pb(v);
        adj[v].pb(u);
    }

    void dfs(int u, int p = -1) {
        in_time[u] = ++timer;
        flat_tree.pb(u);
        for (int v : adj[u]) {
            if (v != p) dfs(v, u);
        }
        out_time[u] = timer;
    }

    void build(int root = 1) {
        timer = 0;
        flat_tree.clear();
        dfs(root);
    }

    // Returns [L, R] 1-based index range for subtree queries on node u
    pair<int, int> get_subtree_range(int u) const {
        return {in_time[u], out_time[u]};
    }
};
```

10 Game Theory

11 Geometry

11.1 Geometry 2D

11.1.1 Point2D

```
// 2D Point / Vector Primitive
template <typename T = double>
struct Point {
    typedef Point P;
    T x, y;

    Point(T x = 0, T y = 0) : x(x), y(y) {}

    bool operator<(P p) const { return tie(x, y) < tie(p.x, p.y); }

    bool operator==(P p) const { return tie(x, y) == tie(p.x, p.y); }

    P operator+(P p) const { return P(x + p.x, y + p.y); }

    P operator-(P p) const { return P(x - p.x, y - p.y); }

    P operator*(T d) const { return P(x * d, y * d); }

    P operator/(T d) const { return P(x / d, y / d); }

    T dot(P p) const { return x * p.x + y * p.y; }

    T cross(P p) const { return x * p.y - y * p.x; }

    T cross(P a, P b) const { return (a - *this).cross(b - *this); }

    T dist2() const { return x * x + y * y; }

    T dist2(P b) const { return (*this - b).dist2(); };

    double dist() const { return sqrt((double) dist2()); }

    double dist(P b) { return (*this - b).dist(); };

    // angle to x-axis in interval [-pi, pi]
    double angle() const { return atan2(y, x); }

    P unit() const { return *this / dist(); } // makes dist()=1
    P perp() const { return P(-y, x); } // rotates +90 degrees
    P normal() const { return perp().unit(); }

    P getVector(const P &p) const { return p - (*this); }

    // returns point rotated 'a' radians ccw around the origin
    P rotate(double a) const {
        return P(x * cos(a) - y * sin(a), x * sin(a) + y * cos(a));
    }

    friend ostream &operator<<(ostream &os, P p) {
//         return os << "(" << p.x << "," << p.y << ")";
        return os << p.x << " " << p.y;
    }

    friend istream &operator>>(istream &is, P &p) {
        return is >> p.x >> p.y;
    }


    // Project point onto line through a and b (assuming a != b).
    P projectOnLine(const P &a, const P &b) const {
        P ab = a.getVector(b);
        P ac = a.getVector(*this);
        return a + ab * ac.dot(ab) / a.dist2(b);
    }

    // Project point c onto line segment through a and b (assuming a !=
        b).
    P projectOnSegment(const P &a, const P &b) const {
        P ab = b - a, ac = *this - a;
        long double r = ac.dot(ab), d = a.dist2(b);
        if (r < 0) return a;
        if (r > d) return b;
        return a + ab * r / d;
    }

    P reflectAroundLine(const P &a, const P &b) const {
        return projectOnLine(a, b) * 2 - *this;
    }
};

int dcmp(const ld &a, const ld &b){
    if(fabs(a - b) < eps)
        return 0;

    return (a > b ? 1 : -1);
}
```

11.1.2 Line2D

```
template<class T>
double lineDist(T p, T s, T e) {
    if (s == e) {
        return s.dist(p);
    }
    return fabs((p - s).cross(e - s) / (e - s).dist());
}

template<class T>
pair<int, T> lineInter(T s1, T e1, T s2, T e2) {
    // first = 0 no intersection
    // first = 1 intersection
    // first = -1 infinite intersection
    auto d = (e1 - s1).cross(e2 - s2);
    if (dcmp(d,0) == 0) // if parallel   same line first = -1 else
        first = 0
        return {-(s1.cross(e1, s2) == 0), {0,0}};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}



template<class T>
bool onSegment(T p,T s, T e) {
    return dcmp(p.cross(s, e) ,0) == 0 && dcmp((s - p).dot(e - p) , 0)
        <= 0;
}


template<class T>
double segDist(T p, T s, T e) {
    if (dcmp((p - s).dot(e - s),0) <= 0)return s.dist(p);
    if (dcmp((p - e).dot(e - s),0) >= 0)return e.dist(p);
    return lineDist(p, s, e);
}

template<class T>
pair<int, T> segInter(T s1, T e1, T s2, T e2) {
    // first = 0 no intersection
    // first = 1 intersection`
```

```
    // first = -1 infinite intersection
    pair<int, T> ret = lineInter(s1, e1, s2, e2);
    if (ret.first == 0)return ret;
    else if (ret.first == 1) {
        if (onSegment(ret.second, s1, e1) && onSegment(ret.second, s2,
            e2))
            return ret;
        else
            return {0, {0, 0}};
    } else {
        if (onSegment(s1, s2, e2) || onSegment(e1, s2, e2))
            return {-1, (onSegment(s1, s2, e2) ? s1 : e1)};
        else if (onSegment(s2, s1, e1) || onSegment(e2, s1, e1))
            return {-1, (onSegment(s2, s1, e1) ? s2 : e2)};
        else
            return {0, {0, 0}};
    }
}


template<class T>
T closestOnSegment(T p, T s, T e) {
    if ((p - s).dot(e - s) <= 0) return s;
    else if ((p - e).dot(e - s) >= 0) return e;
    else return p.projectOnLine(s, e);
}

template<class T>
double segSegDist(T s1, T e1, T s2, T e2) {
    if(segInter(s1,e1,s2,e2).first != 0)
        return 0;
    double ret = min({segDist(s1,s2,e2), segDist(e1,s2,e2), segDist(s2,
         s1,e1), segDist(e2,s1,e1)});
    return ret;
}



template<class T>
bool onRay(T p, T s, T e) {
    return dcmp(p.cross(s, e) , 0) == 0 && dcmp((p - s).dot(e - s) , 0)
          >= 0;
}

template<class T>
double rayDist(T p, T s, T e) {
    if ((p - s).dot(e - s) <= 0) {
        return s.dist(p);
    }
    return lineDist(p, s, e);
}


template<class T>
pair<int, T> rayInter(T s1, T e1, T s2, T e2) {
    // first = 0 no intersection
    // first = 1 intersection
    // first = -1 infinite intersection
    pair<int, T> ret = lineInter(s1, e1, s2, e2);
    if (ret.first == 0)return ret;
    else if (ret.first == 1) {
        if (onRay(ret.second, s1, e1) && onRay(ret.second, s2, e2))
            return ret;
        else
            return {0, {0,0}};
    } else {
        if(onRay(s1, s2, e2) || onRay(s2, s1, e1))
            return {-1,onRay(s1, s2, e2) ? s1:s2};
        else
            return {0, {0,0}};
    }
}

template<class T>
double rayRayDist(T s1, T e1, T s2, T e2) {
    if(rayInter(s1,e1,s2,e2).first != 0)
        return 0;
    double ret = min(rayDist(s1,s2,e2), rayDist(s2,s1,e1));
    return ret;
}
```

11.1.3 AngleOperations

```
// Angle Operations in 2D - [-pi, pi] and [0, 2*pi]
template<typename T>
ld angleBetween(T a, T b) {
    ld res = atan2(a.cross(b), a.dot(b));
    return res < 0 ? res + 2 * PI : res;
}

template<typename T>
ld angle0(T a, T 0, T b) {
    return angleBetween(a - 0, b - 0);
}

ld formatAngle(ld angle) {
    while (angle > PI) angle -= 2 * PI;
    while (angle <= -PI) angle += 2 * PI;
    return angle;
}
```

11.1.4 PolygonArea

```
// Polygon Area (Shoelace Formula) - O(N)
template<typename T>
ld polygonArea(const vector<T>& v) {
    ld area = 0;
    int n = v.size();
    for (int i = 0; i < n; i++) {
        area += v[i].cross(v[(i + 1) % n]);
    }
    return 0.5 * abs(area);
}
```

11.1.5 Circle3Points

```
// Circumcenter & Circumradius of 3 Points
typedef Point<double> P;

bool isColliner(const P &A, const P &B, const P &C) {
    return dcmp(P(B - A).cross(P(C - A)), 0) == 0;
}

P ccCenter(const P &A, const P &B, const P &C) {
    P b = C - A, c = B - A;
    return A + (b * c.dist2() - c * b.dist2()).perp() / b.cross(c) / 2;
}

double ccRadius(const P &A, const P &B, const P &C) {
    return (B - A).dist() * (C - B).dist() * (A - C).dist() / (2 * abs
         ((B - A).cross(C - A)));
}
```

11.1.6 CircleLineIntersection

```
template<class T>
vector<T> circleLine(T o, double r, T a, T b) {
    double h2 = r * r - lineDist(o, a, b) * lineDist(o, a, b);
    if (dcmp(h2, 0) != -1) { // the line touches the circle
        T p = o.projectOnLine(a, b); // point P
        T h = (a - b).unit() * sqrt(h2); // vector parallel to l, of
             length h
        if (p - h == p + h)
            return {p - h};
        else
            return {p - h, p + h};
    }
    return {};
}

template<class T>
vector<T> circleSegment(T o, double r, T a, T b) {
    vector<T> v = circleLine(o, r, a, b);
    vector<T> out;
    for (auto x: v)
        if (onSegment(x, a, b))
            out.pb(x);
    return out;
}
```

11.1.7 CircleCircleIntersection

```
template<class T>
ld circleInter(T a,T b,ll r1,ll r2){
    if(r1 > r2){
        swap(r1,r2);
        swap(a,b);
    }
    ld d = sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
    if(d >= r1 + r2){
        return 0;
    }
    if(d + r1 <= r2){
        return PI * r1 * r1;
    }
    ld theta = acos((d * d + r2*r2 - r1*r1)/(2 * d * r2));
    ld alpha = acos((d * d + r1*r1 - r2*r2)/(2 * d * r1));
    ld s1 = theta * r2 * r2;
    ld t1 = 0.5 * r2 * r2 * sin(2 * theta);
    ld s2 = alpha * r1 * r1;
    ld t2 = 0.5 * r1 * r1 * sin(2 * alpha);
    return (s1 - t1) + (s2 - t2);
}
```

11.1.8 ConvexHull

```
// Convex Hull (Andrew's Monotone Chain) - O(N log N)
// Returns vertices in counter-clockwise order
template<typename P>
vector<P> convexHull(vector<P> pts) {
    int n = pts.size(), k = 0;
    if (n <= 2) return pts;
    vector<P> h(2 * n);
    sort(all(pts));
    for (int i = 0; i < n; i++) {
        while (k >= 2 && h[k - 2].cross(h[k - 1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; i--) {
        while (k >= t && h[k - 2].cross(h[k - 1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    h.resize(k - 1);
    return h;
}
```

11.1.9 RotatingCalipers

```
// Rotating Calipers - O(N) Convex Polygon Diameter & Width
template<typename T>
ll hullDiameter(const vector<T>& S) {
    int n = S.size(), j = n < 2 ? 0 : 1;
    ll res = 0;
    for (int i = 0; i < j; i++) {
        for (;; j = (j + 1) % n) {
            res = max(res, (ll)(S[i] - S[j]).dist2());
            if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
                 break;
        }
    }
    return res;
}

template<typename T>
ld hullWidth(const vector<T>& S) {
    int n = S.size();
    if (n <= 2) return 0;
```

```
    int j = 1;
    ld res = 1e18;
    for (int i = 0; i < n; i++) {
        while ((S[(i + 1) % n] - S[i]).cross(S[(j + 1) % n] - S[j]) >=
             0) j = (j + 1) % n;
        res = min(res, lineDist(S[j], S[i], S[(i + 1) % n]));
    }
    return res;
}
```

11.1.10 MinimumEnclosingCircle

```
// Minimum Enclosing Circle (Welzl's Algorithm) - O(N) Expected Time
typedef Point<double> P;

P ccCenter(const P& A, const P& B, const P& C) {
    P b = C - A, c = B - A;
    return A + (b * c.dist2() - c * b.dist2()).perp() / b.cross(c) / 2;
}

pair<P, double> mec(vector<P> ps) {
    shuffle(all(ps), mt19937(chrono::steady_clock::now().
         time_since_epoch().count()));
    P o = ps[0];
    double r = 0, EPS = 1 + 1e-9;
    int n = ps.size();
    for (int i = 0; i < n; i++) {
        if ((o - ps[i]).dist() > r * EPS) {
            o = ps[i]; r = 0;
            for (int j = 0; j < i; j++) {
                if ((o - ps[j]).dist() > r * EPS) {
                    o = (ps[i] + ps[j]) / 2;
                    r = (o - ps[i]).dist();
                    for (int k = 0; k < j; k++) {
                        if ((o - ps[k]).dist() > r * EPS) {
                            o = ccCenter(ps[i], ps[j], ps[k]);
                            r = (o - ps[i]).dist();
                        }
                    }
                }
            }
        }
    }
    return {o, r};
}
```

11.1.11 HalfPlaneIntersection

```
// Basic half-plane struct.
template<class T>
struct Halfplane {

    // 'p' is a passing point of the line and 'pq' is the direction
         vector of the line.
    T p, pq;
    long double angle;

    Halfplane() {}
    Halfplane(const T& a, const T& b) : p(a), pq(b - a) {
        angle = atan2l(pq.y, pq.x);
    }

    // Check if point 'r' is outside this half-plane.
    // Every half-plane allows the region to the LEFT of its line.
    bool out(const T& r) {
        return dcmp(pq.cross(r - p),0) < 0;
    }

    // Comparator for sorting.
    bool operator < (const Halfplane& e) const {
        return angle < e.angle;
    }

    // Intersection point of the lines of two half-planes. It is
         assumed they're never parallel.
    friend T inter(const Halfplane& s, const Halfplane& t) {
        long double alpha = (ld)(t.p - s.p).cross(t.pq) / s.pq.cross(t.
             pq);
        return s.p + (s.pq * alpha);
    }
};

template<class T>
vector<T> hpIntersect(vector<Halfplane<T>> H) {
    ld inf = 1e9;
    T box[4] = {  // Bounding box in CCW order
            Point(inf, inf),
            Point(-inf, inf),
            Point(-inf, -inf),
            Point(inf, -inf)
    };

    for(int i = 0; i < 4; i++) { // Add bounding box half-planes.
        Halfplane aux(box[i], box[(i+1) % 4]);
        H.push_back(aux);
    }

    // Sort by angle and start algorithm
    sort(H.begin(), H.end());
    deque<Halfplane<T>> dq;
    int len = 0;
    for(int i = 0; i < int(H.size()); i++) {

        // Remove from the back of the deque while last half-plane is
             redundant
        while (len > 1 && H[i].out(inter(dq[len-1], dq[len-2]))) {
            dq.pop_back();
            --len;
        }

        // Remove from the front of the deque while first half-plane is
             redundant
        while (len > 1 && H[i].out(inter(dq[0], dq[1]))) {
            dq.pop_front();
            --len;
        }

        // Special case check: Parallel half-planes
        if (len > 0 && fabsl(H[i].pq.cross(dq[len-1].pq)) < eps) {
            // Opposite parallel half-planes that ended up checked
                 against each other.
            if (dcmp(H[i].pq.dot(dq[len-1].pq),0) < 0)
                return vector<T>();

            // Same direction half-plane: keep only the leftmost half-
                 plane.
            if (H[i].out(dq[len-1].p)) {
                dq.pop_back();
                --len;
            }
            else continue;
        }

        // Add new half-plane
        dq.push_back(H[i]);
        ++len;
    }

    // Final cleanup: Check half-planes at the front against the back
         and vice-versa
    while (len > 2 && dq[0].out(inter(dq[len-1], dq[len-2]))) {
        dq.pop_back();
        --len;
    }

    while (len > 2 && dq[len-1].out(inter(dq[0], dq[1]))) {
        dq.pop_front();
        --len;
    }

    // Report empty intersection if necessary
    if (len < 3) return vector<T>();

    // Reconstruct the convex polygon from the remaining half-planes.
    vector<T> ret(len);
    for(int i = 0; i < len; i++) {
        ret[i] = inter(dq[i], dq[(i+1)%len]);
    }
    return ret;
}
```

11.2 Geometry 3D

11.2.1 Point3D

```
// 3D Point / Vector Primitive
template <typename T = double>
struct Point3D {
    typedef Point3D P;
    typedef const P &R;
    T x, y, z;

    explicit Point3D(T x = 0, T y = 0, T z = 0) : x(x), y(y), z(z) {}

    bool operator<(R p) const {
        return tie(x, y, z) < tie(p.x, p.y, p.z);
    }

    bool operator==(R p) const {
        return tie(x, y, z) == tie(p.x, p.y, p.z);
    }

    P operator+(R p) const { return P(x + p.x, y + p.y, z + p.z); }

    P operator-() const { return P(-x, -y, -z); }

    P operator-(R p) const { return P(x - p.x, y - p.y, z - p.z); }

    P operator*(T d) const { return P(x * d, y * d, z * d); }

    P operator/(T d) const { return P(x / d, y / d, z / d); }

    T dot(R p) const { return x * p.x + y * p.y + z * p.z; }

    P cross(R p) const {
        return P(y * p.z - z * p.y, z * p.x - x * p.z, x * p.y - y * p.
             x);
    }

    T dist2() const { return x * x + y * y + z * z; }

    double dist() const { return sqrt((double) dist2()); }

    //Azimuthal angle ( longitude) to x=axis in interval [=pi , pi ]
    double phi() const { return atan2(y, x); }

    //Zenith angle ( latitude ) to the z=axis in interval [0 , pi ]
    double theta() const { return atan2(sqrt(x * x + y * y), z); }

    P unit() const { return *this / (T) dist(); } //makes dist ()=1
    //returns unit vector normal to *this and p
    P normal(P p) const { return cross(p).unit(); }

    //returns point rotated 'angle ' radians ccw around axis
    P rotate(double angle, P axis) const {
        double s = sin(angle), c = cos(angle);
        P u = axis.unit();
        return u * dot(u) * (1 - c) + (*this) * c - cross(u) * s;
    }

    friend ostream &operator<<(ostream &os, P p) {
//         return os << "(" << p.x << "," << p.y << ")";
        return os << p.x << " " << p.y << " " << p.z;
    }

    friend istream &operator>>(istream &is, P &p) {
        return is >> p.x >> p.y >> p.z;
    }
```

```
};


namespace Axis {
    Point3D<double> x(1, 0, 0);
    Point3D<double> y(0, 1, 0);
    Point3D<double> z(0, 0, 1);
};
```